



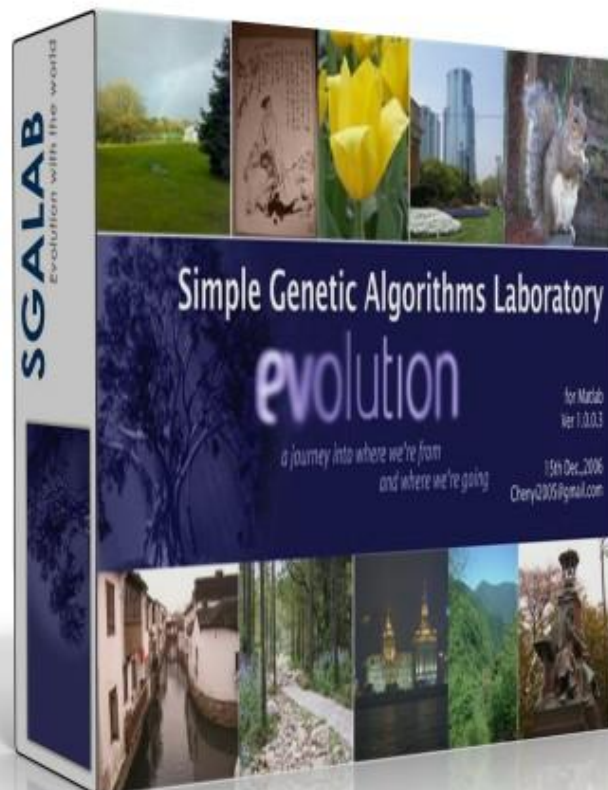
SGALAB User's Guide & FAQ

Chen , Yi
chenyi2005@gmail.com
10th Dec.,2006

(Not full Version)

Revision History

Doc Rev.#	Date of change	Name of person making change	Description of Change	SGALAB Version
1.0	01-Nov-2006	Chen,Yi	Initial version	1.0.0.3 Beta4
2.0	10-Dec-2006	Chen,Yi	Add NSGA-II	1.0.0.3 Beta5
3.0	22-Feb-2007	Chen,Yi	Add encoding fig.	1.0.0.3 Beta6



Preface

Genetic Algorithms (**GAs or GA**) was formally introduced in the United States in the 1970s by John Holland at University of Michigan. It is inspired by Darwin's theory about evolution. Solution to a problem solved by GAs is evolved. GAs has been widely studied, experimented and applied in many fields in engineering worlds. Not only does GAs provide a group of alternative methods to solve problems, but also it consistently outperforms other traditional methods in most of the problems. Many of the real world problems involved finding optimal parameters, which might prove difficult for traditional methods but ideal for GAs. However, because of its outstanding performance in optimization, GAs has been wrongly regarded as a family of function optimizers. In particular, genetic algorithms works very well on mixed (continuous and discrete), combinatorial problems. GAs is started with a set of solutions (represented by chromosomes) called population. Solutions from one population are taken and used to form a new population. This is motivated by a hope, that the new population will be better than the old one. Solutions which are selected to form new solutions (offspring) are selected according to their fitness - the more suitable they are the more chances they have to produce. This is repeated until some condition (for example number of populations or improvement of the best solution) is satisfied. GAs is less susceptible to getting 'stuck' at local optima than gradient search methods, but it tends to be computationally expensive.

To use the genetic algorithms, you need to represent a solution to your problem as a genome (or chromosome). The genetic algorithms can then create a population of solutions and applies genetic operators such as mutation and crossover to evolve the solutions in order to find the best one(s). This presentation outlines some of the basics of genetic algorithms. The three most important aspects of using genetic algorithms are: (1) definition of the objective function (Fitness Function); (2) definition and implementation of the genetic representation (En/De-coding Method); (3) definition and implementation of the genetic operators. Once these three have been defined, the generic genetic algorithm should work fairly well. Beyond that you can try many different variations to improve performance, find multiple optima (species - if they exist), or parallelize the algorithms.

Simple Genetic Algorithms Laboratory (SGALAB) Toolbox for Matlab is a generic toolbox of genetic algorithms. The first version of SGALAB was born in April, 2002, when the author was still a post-graduate student in the State Key Laboratory of Mechanical Transmission of Chongqing University. His interests on the Genetic Algorithms, Fuzzy Logic Control, Artificial Neural Network or some related Intelligent Algorithms was and is pushing him to update the SGALAB toolbox, even now. Yes, it is a hard work needs much efforts and it also thanks to the SGALAB to give the author a clue to insist on his study & research.

During this period, the author wrote a poem as a motto for himself to encourage him to keep an evolutionary & adaptive attitude like Genetic Algorithms in his everyday life. Now, the author gave the 2 poems to those people whose interests are also on genetic algorithms, See the following:

临江仙

陈昔往事益归去，
古道西风依旧。
人生长河水长东，
春去遇秋来，
天际寻鹄鸿。

日月星移话沧桑，
是非功过何在？
一路风雨兼程行，
唯清风明月，
独在情怀中。



蝶恋花

潇潇夜雨润菊篱，
黄叶蝶飞，
老去太容易。
渡河船头愁云散，
数点瘦雁何时还？

纸上墨痕书中意，
推推敲敲，
总是抹不去。
斜月寒窗还少眠，
清风作伴到天明。



Author's Info

Chen Yi is from Chongqing, China. Leo is his English name he gave to himself. He graduated from the State Key Laboratory of Mechanical Transmission, Chongqing University in the July 2004, with the Master Degree, major in Automotive Engineering.

After that, he attended UGS on July 5th, 2004 as a CAE software engineer. And now he is a PhD student in University of Glasgow, Scotland, UK.

If you are interested in Genetic Algorithms or SGALAB Toolbox, we can have a discussion about it, the best place to find leo is in the following forum:

bbs.matwav.com	ID: see_moonlight
bbs.sciei.com	ID: see_moonlight
www.simwe.com	ID: cdey

And you can also contact the author directly in this way:





Main building, University of Glasgow
By Yi Chen
Oct., 2006



A squirrel in Botanic Gardens, Glasgow
By Yi Chen
Nov., 2006

[MATLAB Central](#) > [File Exchange](#) > [Author Index](#) > [Yi Chen](#)

Author Page

Name: Yi Chen
E-mail: leo.chen.yi@gmail.com
Organization: University of Glasgow, James Watt South Building, Glasgow G12 8QQ, Scotland, UK
Rank: 317
Total Downloads: 5857



Author Description

[临江仙]
 陈昔往事益归去，古道西风依旧。人生长河水长东，
 春去遇秋来，天际寻鹤鸿。

日月星移话沧桑，是非功过何在？一路风雨兼程行，
 唯清风明月，独在情怀中。

Files

Click on column heading to sort by column
 ▾ Downloads ▾ Title Submitted Rating (5=best)

Click on column heading to sort by column				
▾ Downloads ▾	Title	Submitted	Rating (5=best)	
4049	SGALAB1003 Beta4.0.0.1 Genetic Algorithm Toolbox Author: Yi Chen Category: Biotech, Pharmaceutical, Medical	2004-09-15	★★★★★ 4.42 23 reviews	
686	2,4,7 DOFs Suspension Model for Simulink 2,4,7 DOFs Suspension Model for Simulink Author: Yi Chen Category: Automotive	2006-05-25	★★★★☆ 3.0 2 reviews	
507	Fuzzy Controller of Semi-active control for 1/4 Suspension System Fuzzy Controller , semi-active Author: Yi Chen Category: Automotive	2006-05-19	★★★★★ 4.5 2 reviews	
337	Semi-active Control of Skyhook for 1/4 Suspension System Semi-active control of skyhook Author: Yi Chen Category: Automotive	2006-05-19	★★★★★ 4.67 3 reviews	
278	SERALAB(Demo)- Simple eigensystem realization algorithm Lab eigensystem realization algorithm (ERA) Author: Yi Chen Category: Automotive	2006-05-24	★★★★★ 5.0 1 review	

Submit a File
 Now accepting MATLAB Published M-files.

- Author Index
- Most Recent Files
- Most Downloaded
- Screenshot Index
- Most Recent Comments
- Files By Product
- Metrics and Reports

Update Author Information
 Notify me when this author submits new file

E-mail:

How do I unsubscribe?

E-mail this page to a colleague

Contents

(Chapter 4 of Genetic Algorithms Basic and its Engineering Applications)

- 4.1 SxLab Family Overview**
- 4.2 Structure of SGALAB**
- 4.3 SGALAB Graphical User Interface(GUI)**
- 4.4 En/De-coding**
 - 4.4.1 Binary En/Decoding**
 - 4.4.2 Permutation En/Decoding**
 - 4.4.3 Gray En/Decoding**
 - 4.4.4 Real&Integer En/Decoding**
 - 4.4.5 Messy En/Decoding**
 - 4.4.6 DNA En/Decoding**
- 4.5 Selection**
 - 4.5.1 Roulette Wheel Selection**
 - 4.5.2 Stochastic Universal Selection**
 - 4.5.3 Tournament Selection**
 - 4.5.4 Truncation Selection**
 - 4.5.5 Uniform Random Selection**
- 4.6 Crossover**
 - 4.6.1 binary single point crossover**
 - 4.6.2 real/integer single point crossover**
 - 4.6.3 TSP – CX**
 - 4.6.4 TSP – PMX**
 - 4.6.5 TSP – OX**
 - 4.6.6 TSP – EAX**
 - 4.6.7 TSP – Topological**
 - 4.6.8 TSP – BoolMatrix**
- 4.7 Mutation**
 - 4.7.1 binary single point mutation**
 - 4.7.2 real/integer single point mutation**
 - 4.7.3 TSP – Displacement**
 - 4.7.4 TSP – Insertion**
 - 4.7.5 TSP – Inversion**
 - 4.7.6 TSP – ReciprocalExchange**
- 4.8 Fitness Function Design**
 - 4.8.1 overview**
 - 4.8.2 fitness scaling /sharing/windowing**
 - 4.8.3 Math Function as Fitness Function**
 - 4.8.4 Table Data as Fitness Function**
 - 4.8.5 Constraint function design**
 - 4.8.6 Bench functions design**
- 4.9 TSP**
 - 4.9.1 TSP path generation**
 - 4.9.2 TSP cost evaluation**

4.9.3 TSP fitness function

4.10 Multi-objective Genetic Algorithms

4.10.1 Sum of Weighted Objectives (Non-Pareto)

4.10.2 Goal Programming (Non-Pareto)

4.10.3 Epsilon Constraint (Non-Pareto)

4.10.4 VEGA (Non-Pareto)

4.10.5 MOGA (Pareto)

4.10.6 NPGA (Pareto)

4.10.7 NSGA (Pareto)

4.10.8 NPGA-II (Pareto)

4.10.9 NSGA-II (Pareto)

4.10.10 SPEA (Pareto)

4.10.11 ELSA (Pareto)

4.10.12 IEDS (Pareto)

4.10.13 MOMGA (Pareto)

4.10.14 MOMGA-II(Pareto)

4.10.15 PAES (Pareto)

4.10.16 IEDS (Pareto)

4.10.17 PDE (Pareto)

4.10.18 MPANN (Pareto)

4.10.19 SPEA2 (Pareto)

4.10.20 PCGA (Pareto)

4.10.21 SPDE (Pareto)

4.10.22 To show all the Multi-objective methods

4.11 Let GAs dance with other CI methods

4.11.1 Genetic Algorithms and Artificial Neural Network

4.11.2 Genetic Algorithms and Fuzzy Logic System

4.11.3 Genetic Algorithms and Ant Algorithm

4.11.4 Genetic Algorithms and Simulated Annealing Algorithm

4.11.5 Virus Evolution Genetic algorithms(VEGA)

4.11.6 Genetic Algorithms and Immune algorithm

4.12 SGALAB FAQ

4.12.1 where can I get SGALAB

4.12.2 set work path for SGALAB

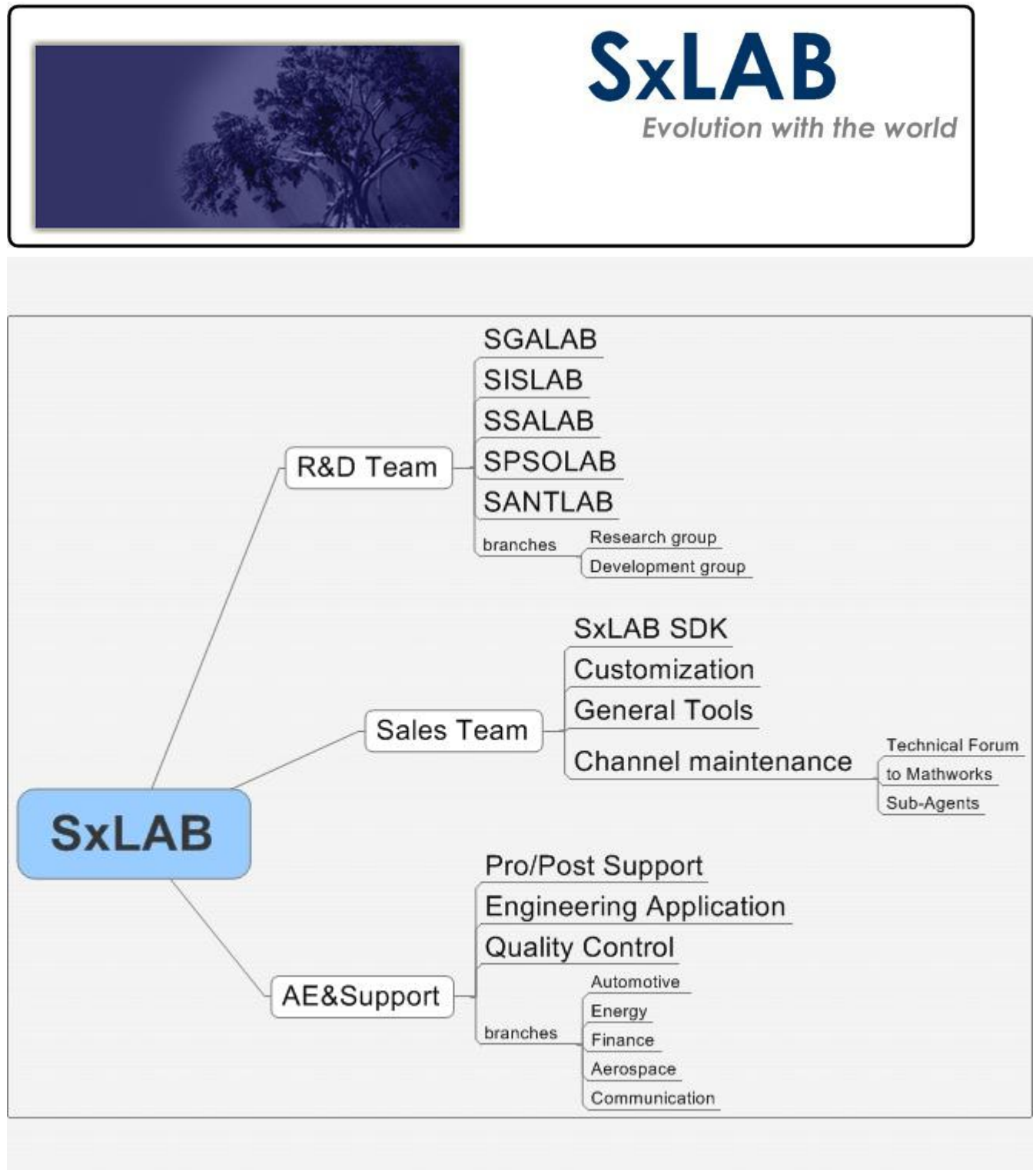
4.12.3 about `randint() `

Appendix

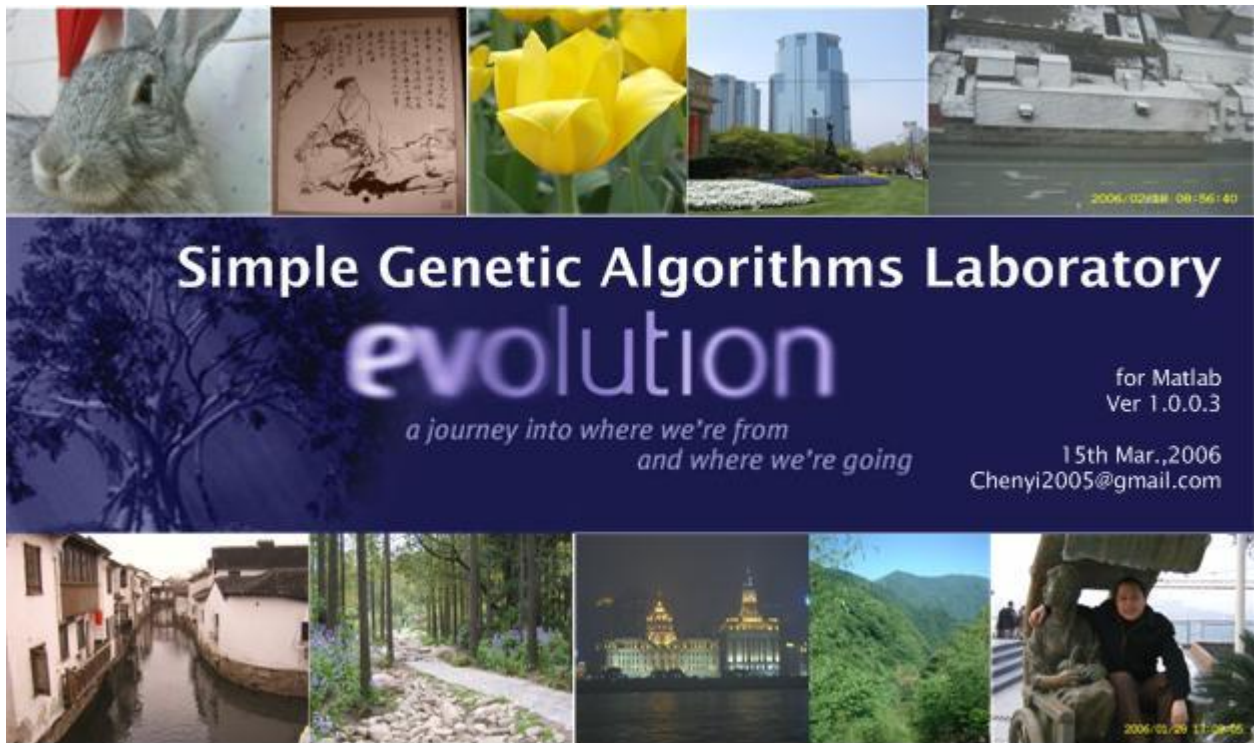
Reference

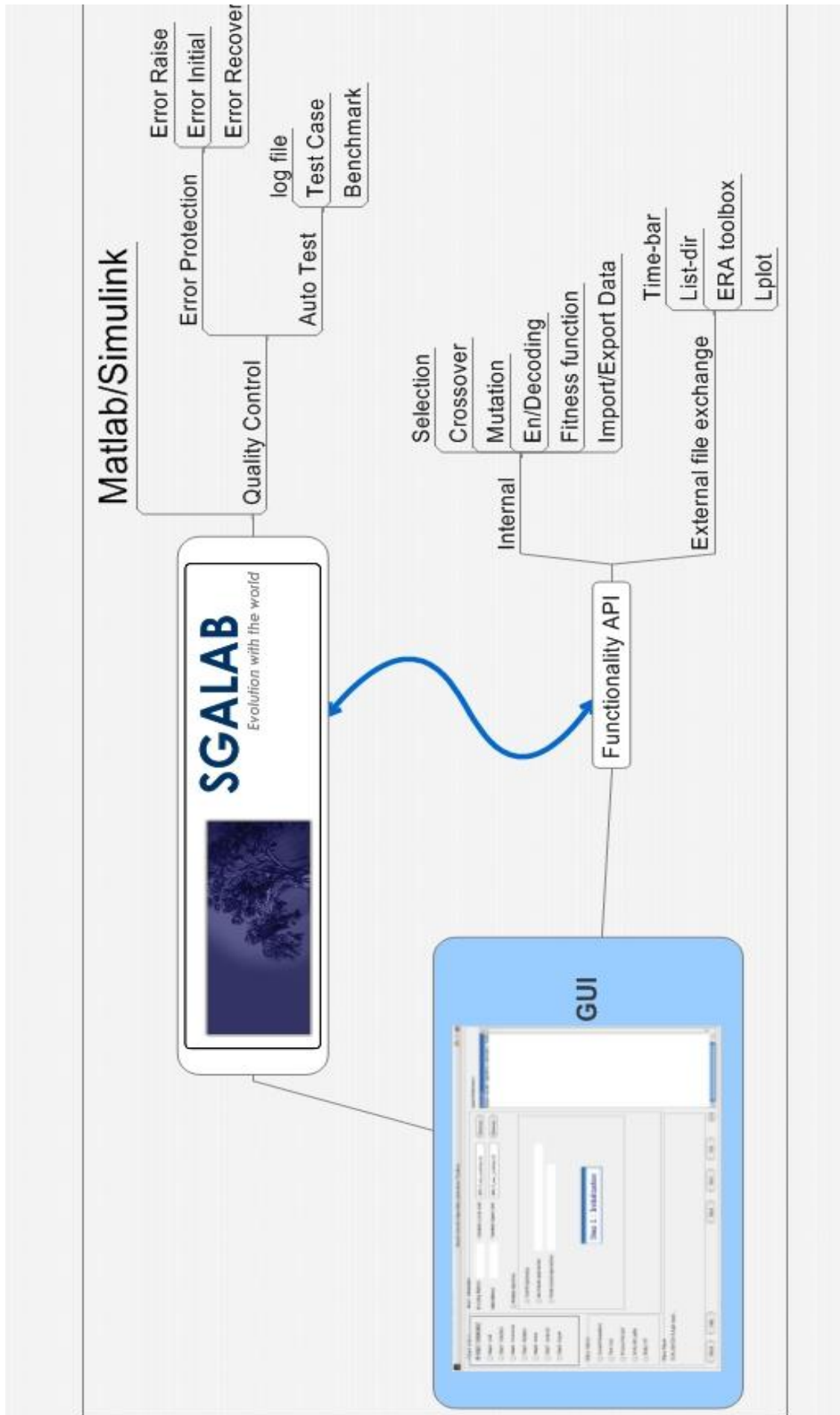
Useful link

4.1 SxLab Family Overview



4.2 Structure of SGALAB





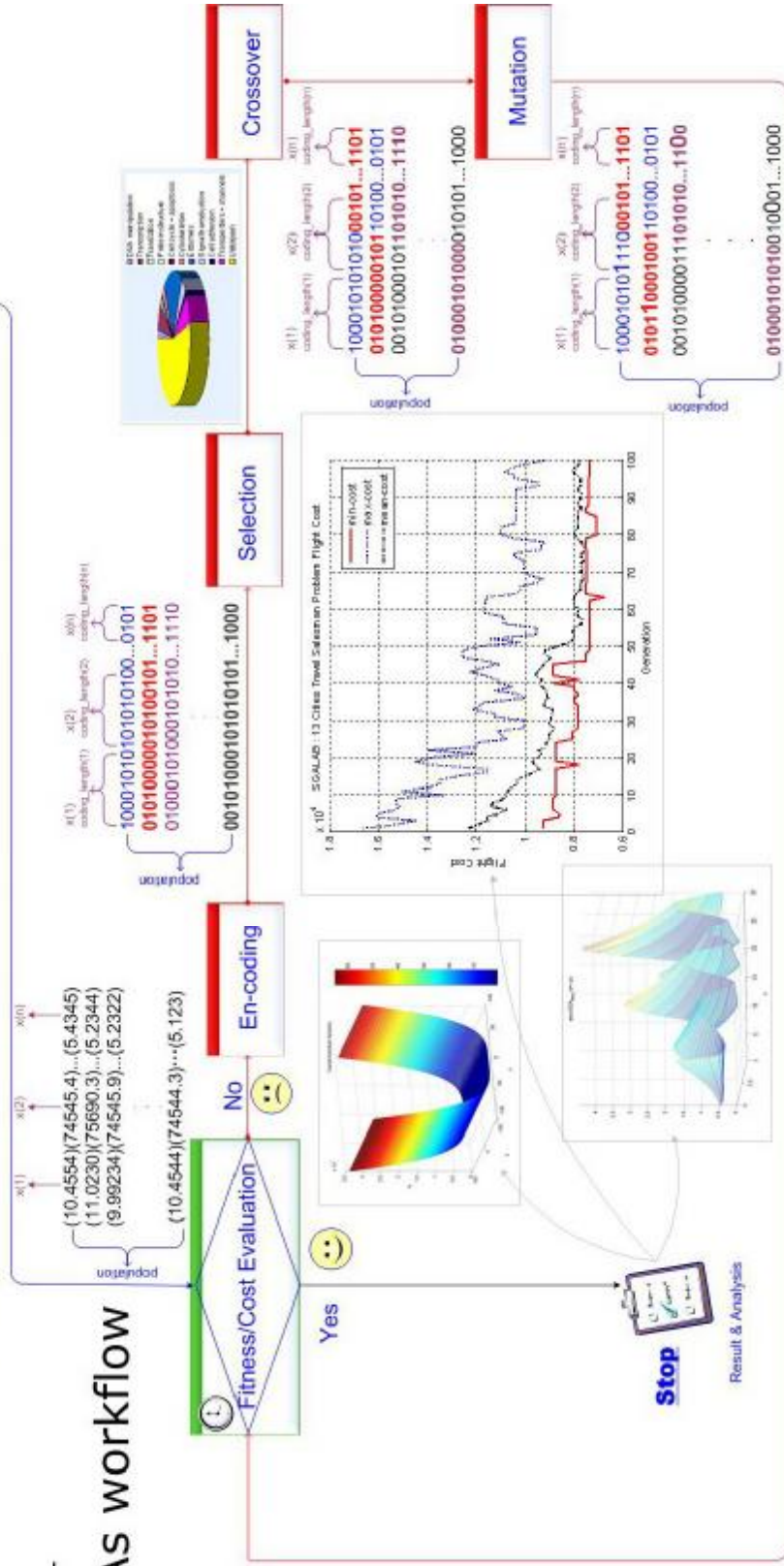
Max : $F(x) = \{ f_1(x), f_2(x), \dots, f_n(x) \}$
 Subject to: $g_j(x) \leq 0$
 $i = 1, 2, \dots, m$
 $j = 1, 2, \dots, n$
 (1) Object design based on engineering application
 (2) Variable/Fitness function design

Start

initial solution

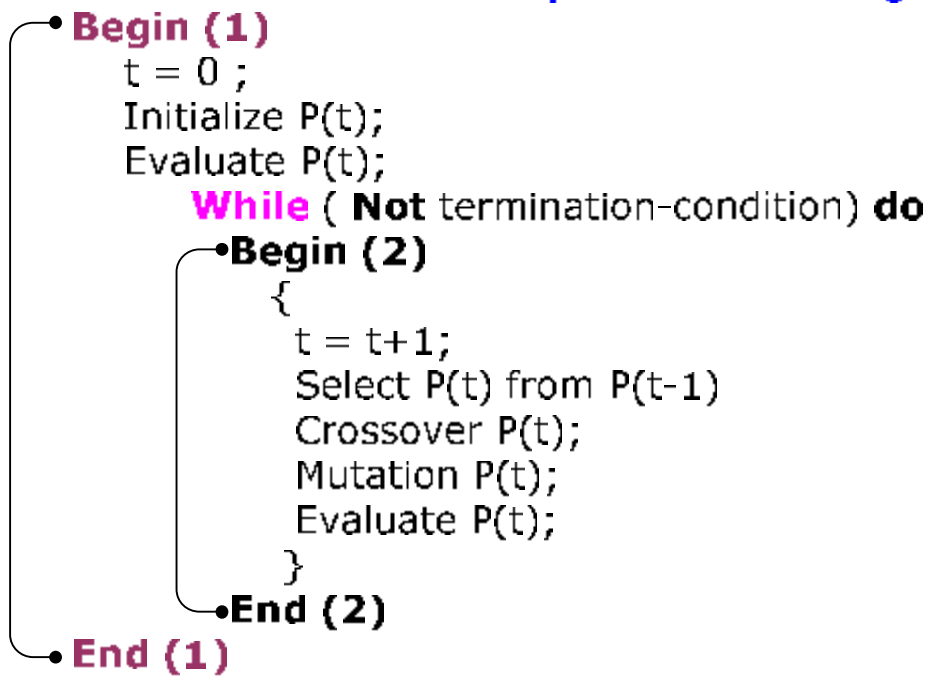
De-coding

SGALAB – Simple GAS workflow

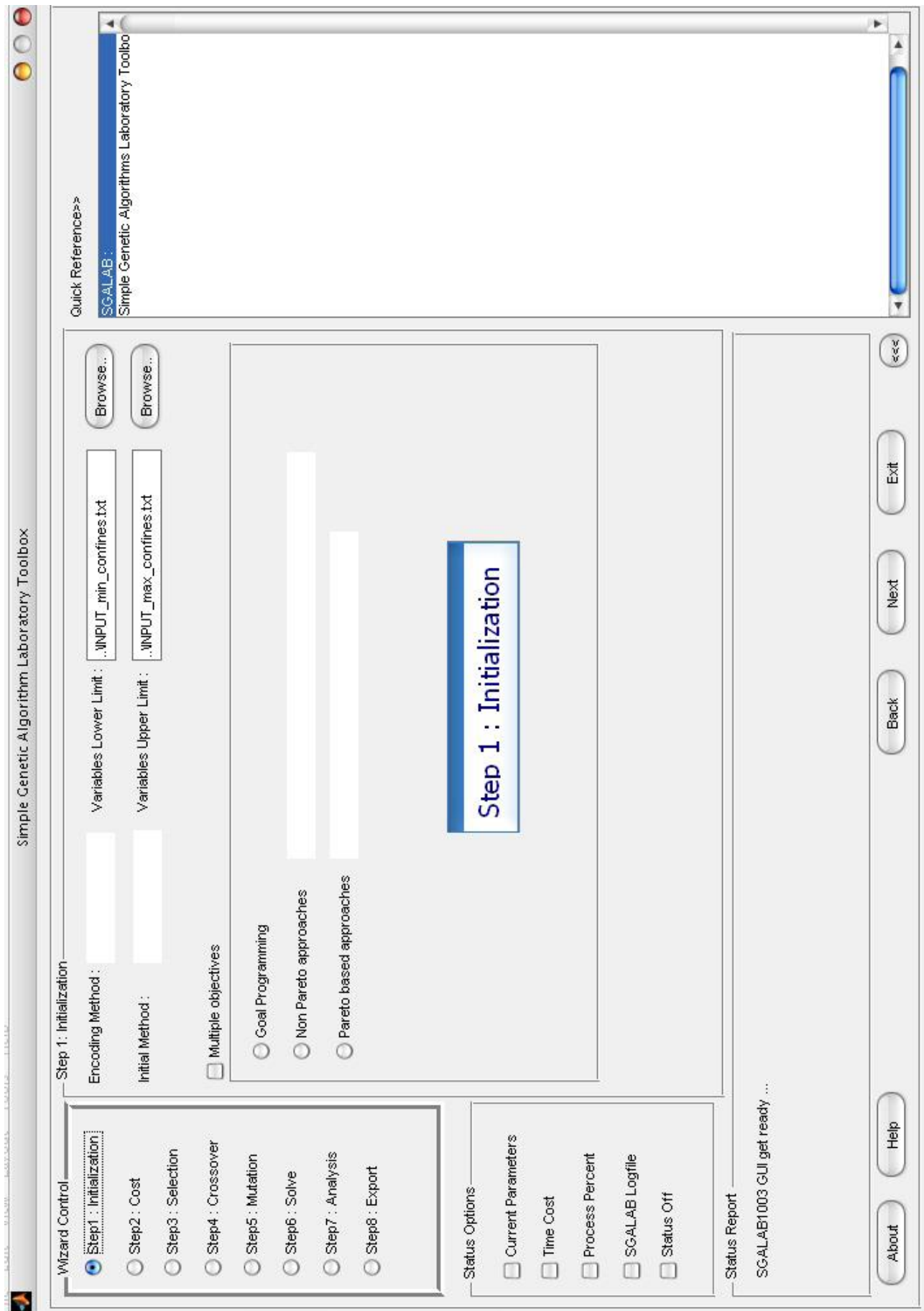


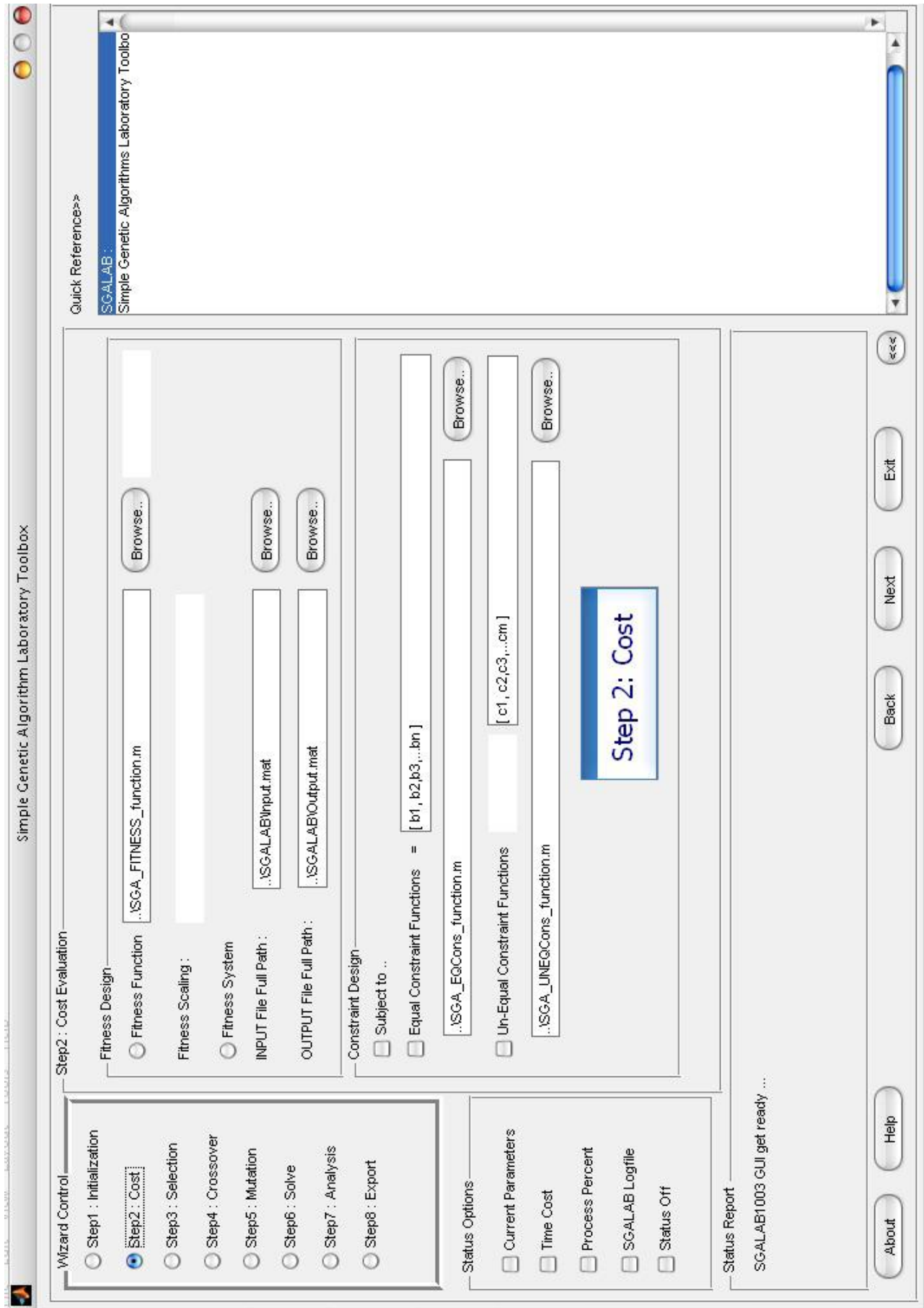
Chen Yi
 chenyl2005@gmail.com
 28th Nov., 2005

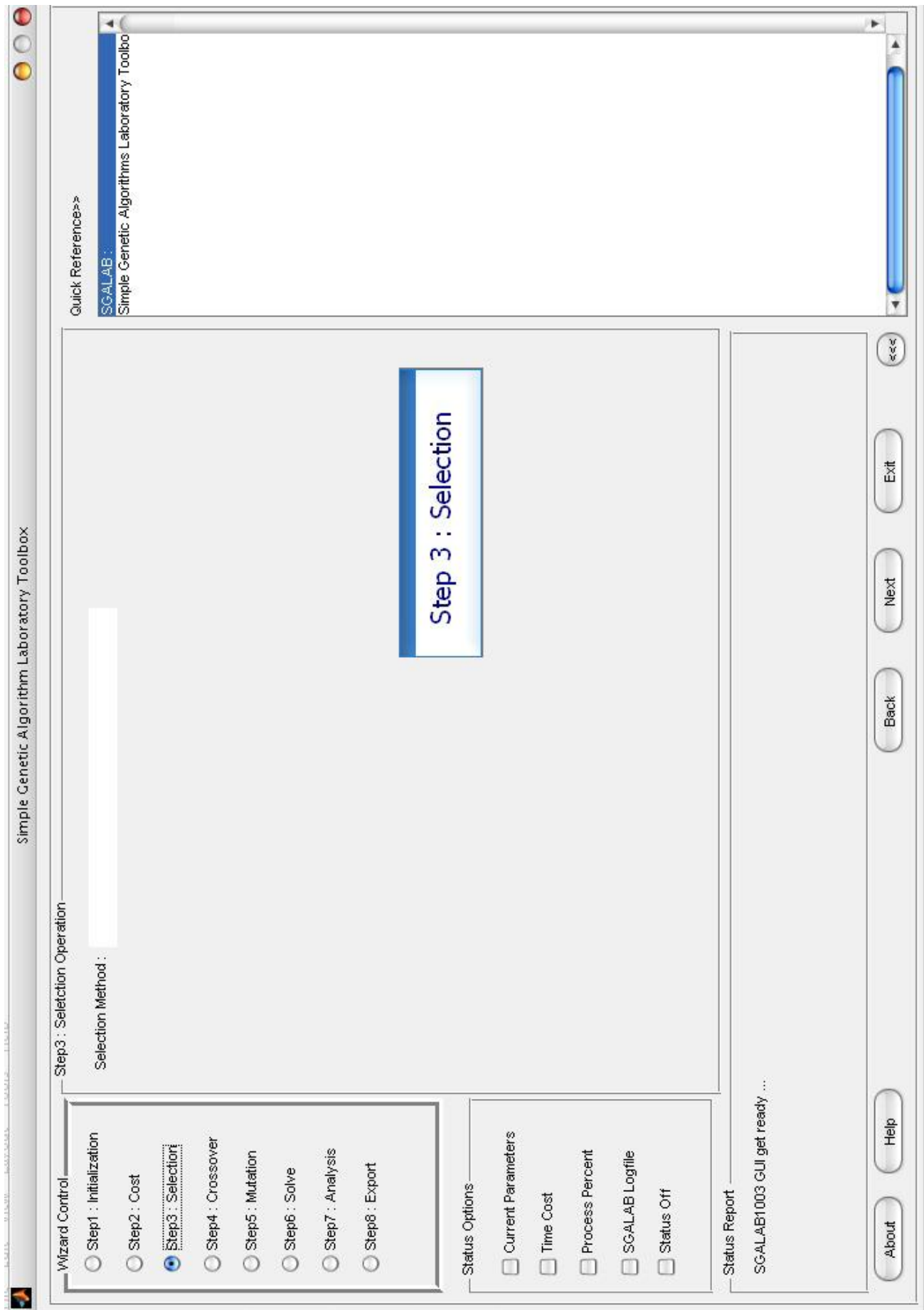
Pseudo Code of Simple Genetic Algorithm

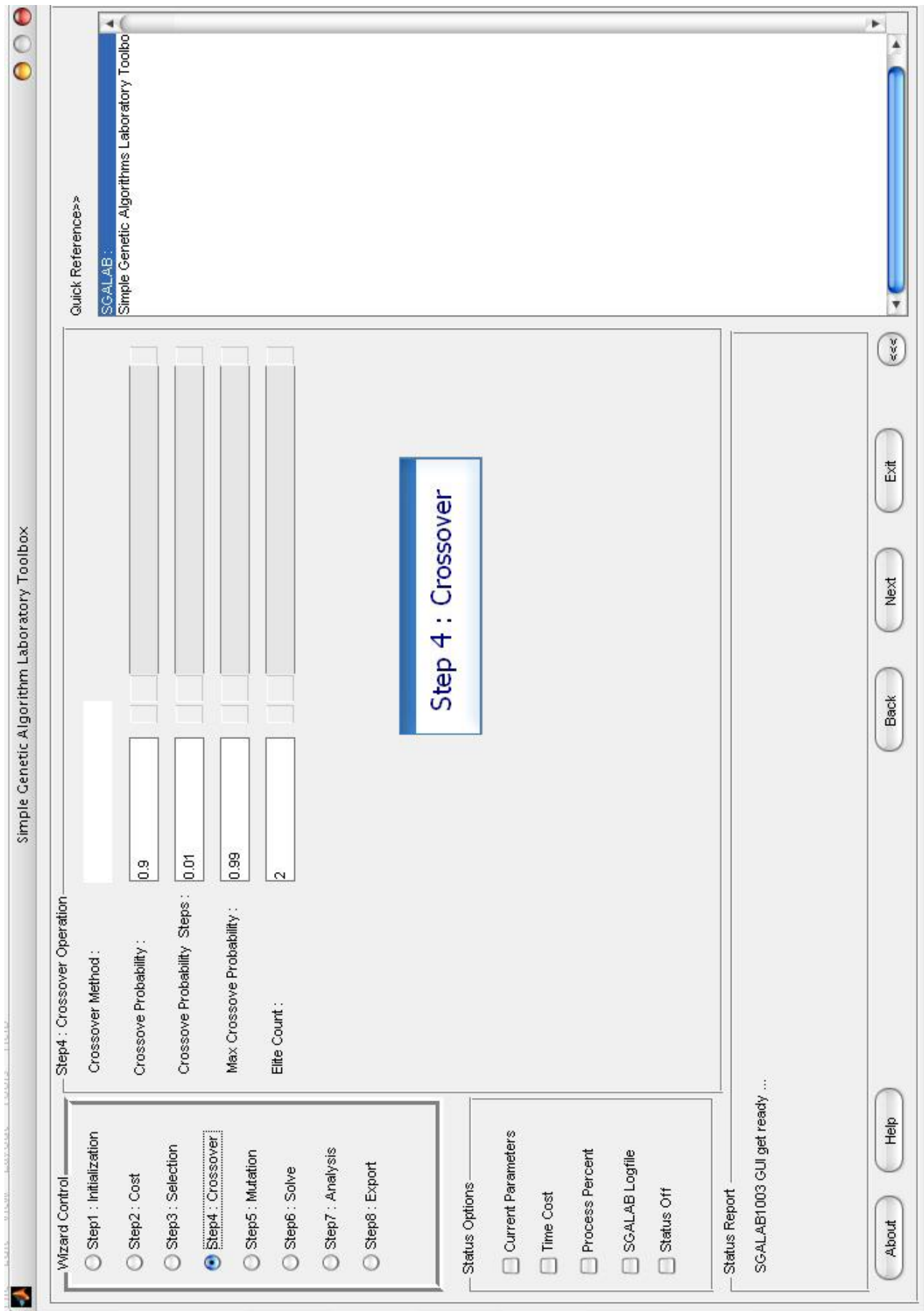


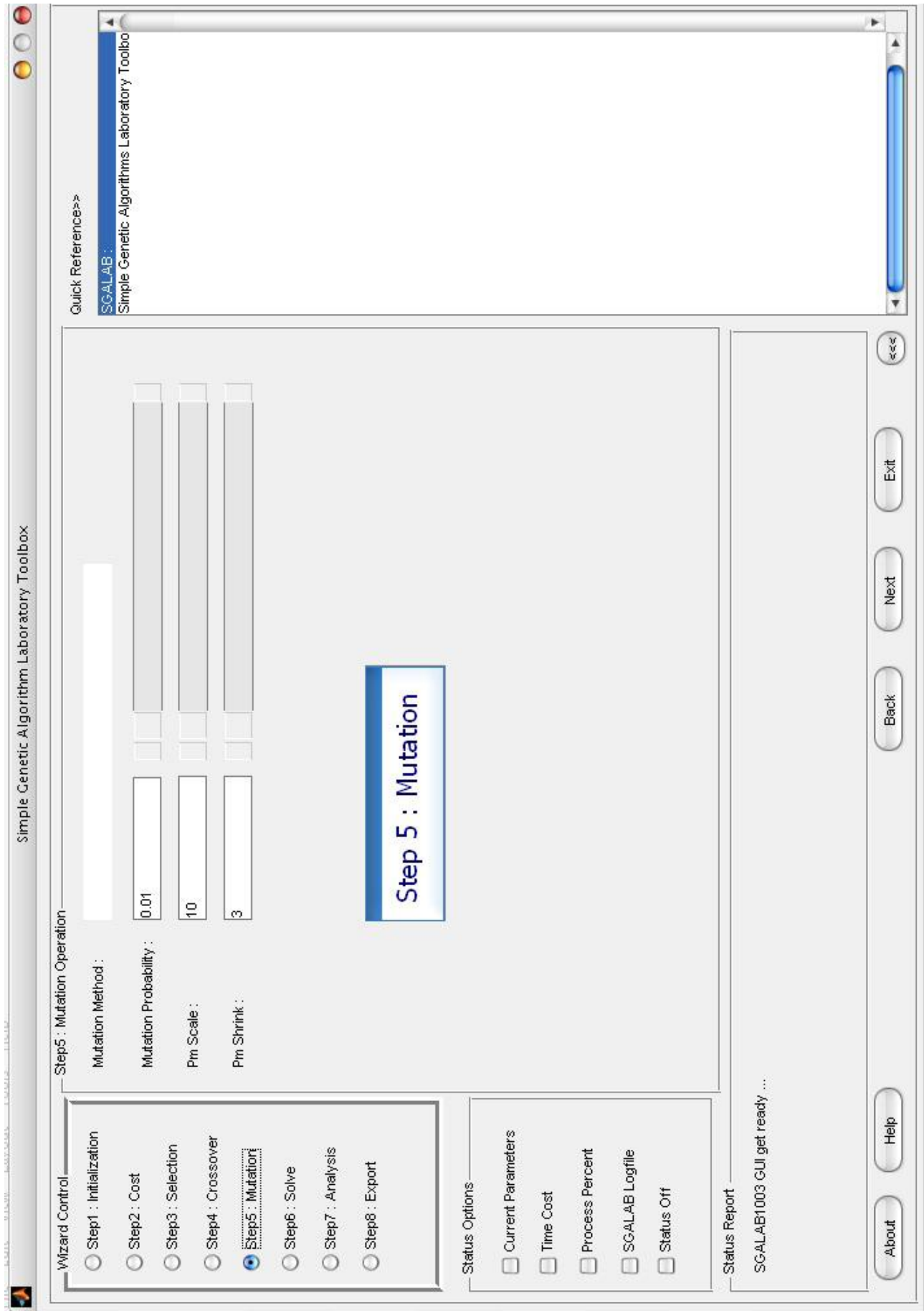
4.3 SGALAB Graphical User Interface(GUI)



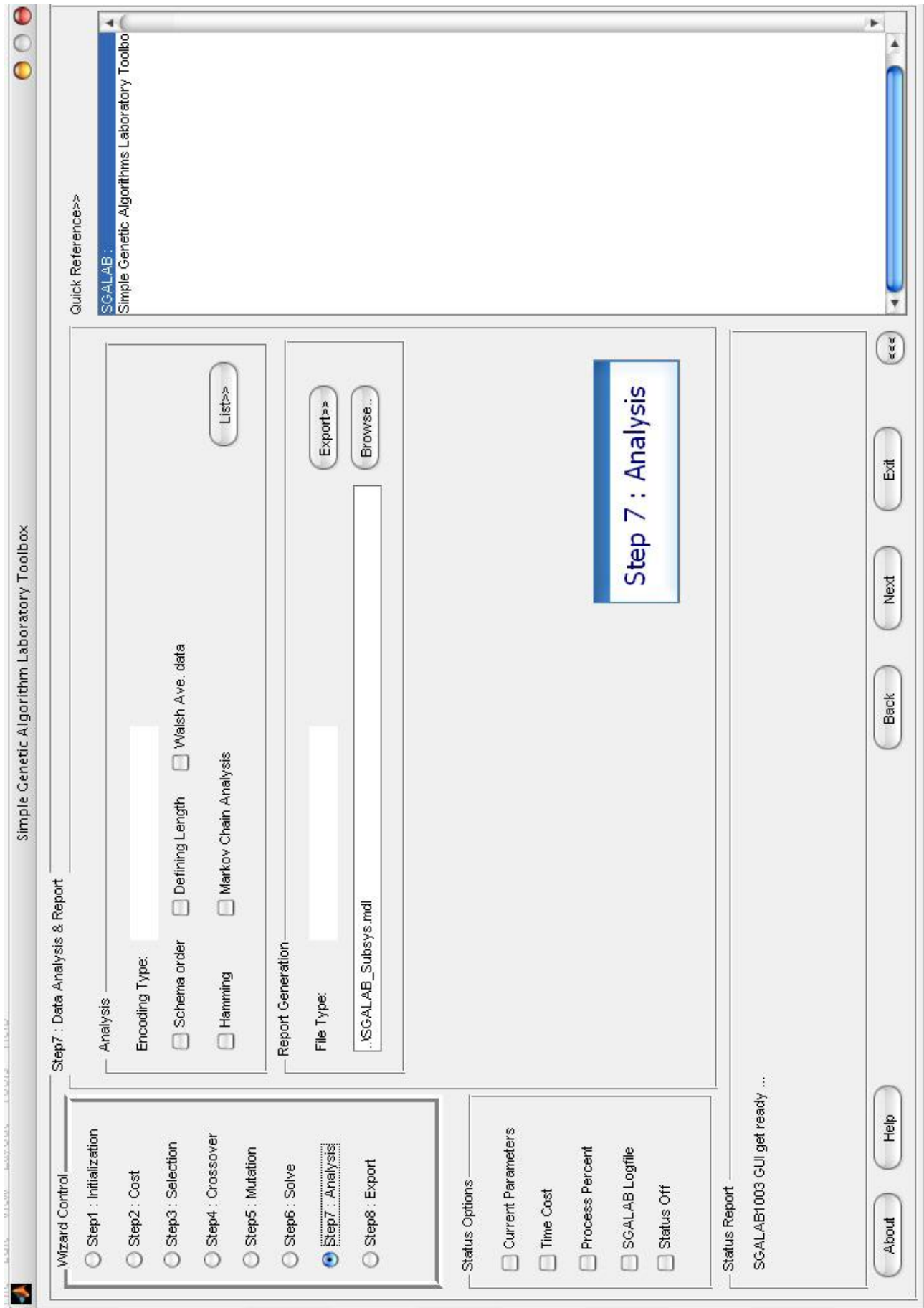


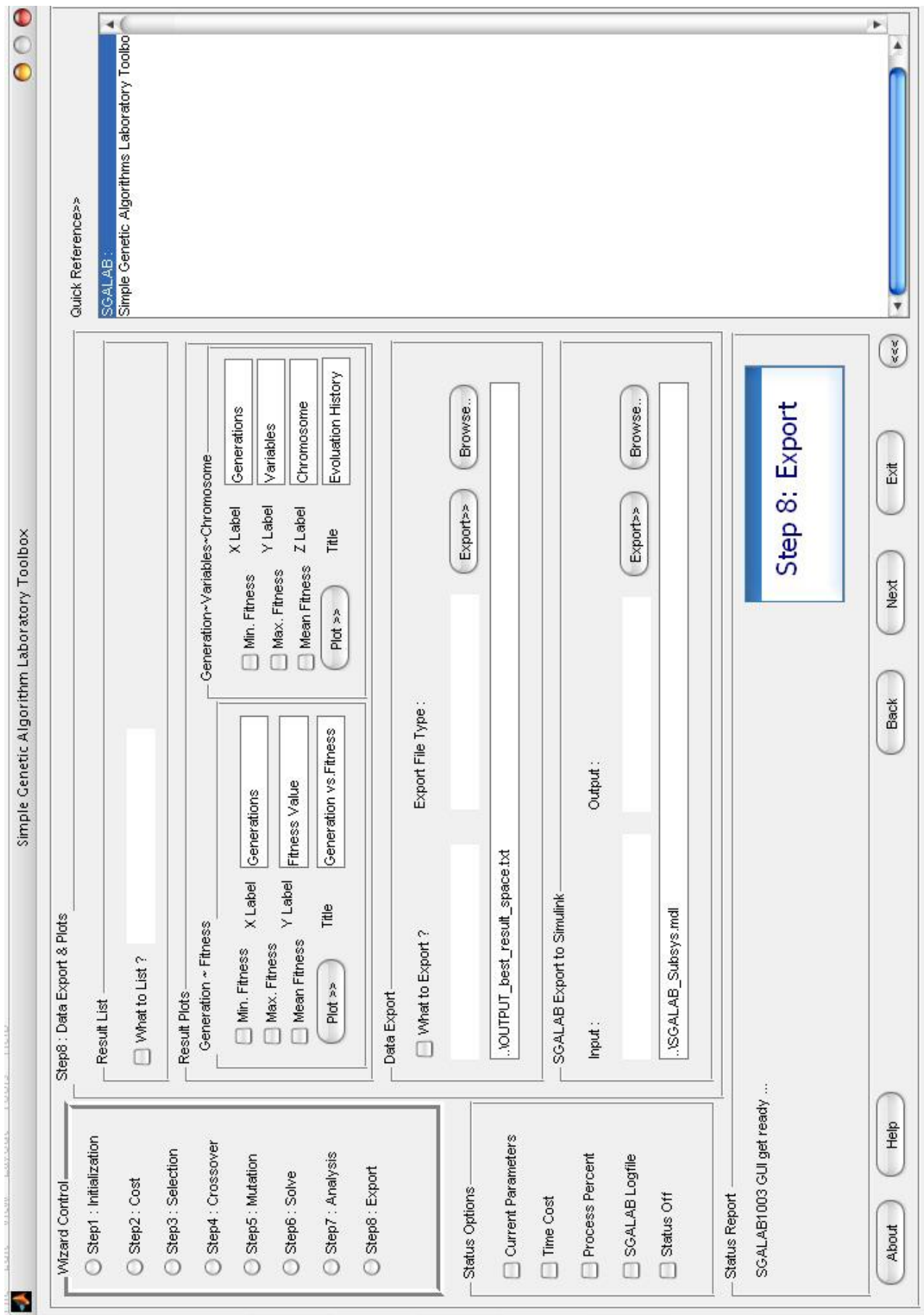












4.4 En/De-coding

SGA_encoding

SGALAB Encoding function

Syntax

```
[ coding_space , bits , error_status ] = SGA_encoding ( options, min_confines ,  
                                                    max_confines , population , decimal_step )
```

Input:

options-- 'Binary' , 'b' : binary encoding method
 'Real' , 'r' : real number encoding method
 'Literal','Permutation','p','l' : literal permutation encoding method
 Permutation encoding for TSP
 Permutation encoding can be used in ordering
 problems, such as the traveling salesman problem
 or a task ordering problem. Every chromosome is
 a string of numbers, which represents number in a
 sequence. In the TSP each number would
 represent a city to be visited.

 'Gray' , 'g' : Gray encoding method
 'DNA' , 'd' : DNA encoding method
 'Messy' , 'm' : Messy encoding method

... [min_confines, max_confines]
min_confines -- min confine for each variable
max_confines -- max confine for each variable
population -- population of individual
decimal_step -- steps for each variables

when TSP:

min_confines -- city_number , e.g. 13 cities

max_confines -- start_city_number, e.g. we want to start from city 4

decimal_step -- no need

Output:

coding_space -- an array contain 1/0 element as a binary string
bits -- the length of every paramete
error_status = 0 -- ok
 1 -- invalid parameter
 2 -- too many input arguments

Description

This is the main entry for encoding operation in SGALAB

Examples

Example1 - binary encoding

[binary_pool, bits, error_status] = SGA_encoding('Binary', [1.1,1],[2,3],20,[0.01,0.01])

binary_pool =

Columns 1 through 14

1	0	1	0	1	1	0	1	1	0	0	0	0	0
0	0	0	1	1	0	0	0	0	0	0	1	0	1
1	1	1	1	1	1	1	1	0	1	0	0	1	1
0	0	0	1	1	0	0	1	0	1	0	1	1	1
1	0	1	1	0	0	1	0	1	0	0	1	1	0
1	0	1	1	1	0	1	0	0	0	1	1	1	1
0	0	1	0	0	1	1	1	0	1	0	0	0	1
0	1	0	0	0	1	1	1	1	0	0	0	0	0
1	0	0	0	1	1	0	0	1	1	0	0	1	1
0	0	0	1	1	1	0	1	1	1	1	0	1	0
1	0	0	1	0	0	0	0	0	0	1	0	0	1
1	1	1	0	0	0	1	0	0	0	0	1	0	1
1	0	0	1	1	1	0	0	0	0	0	0	0	0
1	1	1	1	0	1	1	0	0	0	1	1	1	0
0	0	0	0	0	0	1	0	0	1	0	0	1	1
0	0	1	1	1	1	0	0	1	0	1	0	1	1
1	1	0	1	0	1	1	1	1	1	1	0	1	0
1	1	1	0	0	0	0	0	1	0	1	0	0	0
0	0	1	1	0	1	0	0	0	0	1	1	1	1
1	1	1	1	1	0	1	1	1	1	0	0	1	1

Column 15

1
1
1
0
0
1
1
1
1
0
0
1
0
0
1

1
1
1
1
0

bits =

7 8

error_status =

0

Example2 - Gray encoding

[gray_pool, bits, error_status] = SGA_encoding('gray',[1.1,1],[2,3],6,[0.01,0.01])

gray_pool =

Columns 1 through 14

0	1	0	0	0	1	1	0	0	1	0	1	1	1
1	0	1	1	1	0	1	1	1	0	1	0	0	1
0	1	0	0	0	1	1	1	0	0	0	0	1	0
1	0	0	1	0	1	0	0	1	0	1	0	1	1
0	0	1	0	0	0	0	0	0	1	1	1	0	1
1	0	1	0	0	0	0	0	0	0	0	0	1	1

Column 15

1
0
0
1
0
1

bits =

7 8

error_status =

0

Example3 - real encoding

[coding_space, bits, error_status] = SGA_encoding('real',[0,0],[2,3],10)

coding_space =

1.4481	2.4866
0.5633	0.4988
0.5236	1.1817
1.4169	1.5623
1.5677	2.1544
1.9723	1.7076
0.9467	1.3824
1.8056	1.3359
0.9021	0.2632
1.6090	1.3304

bits =

2

error_status =

0

Example4 - permutation encoding

[coding_space, bits, error_status] = SGA_encoding('permutation',4,6)

coding_space =

2	6	5	1	3	4
4	6	2	5	3	1
4	3	6	1	5	2
1	6	5	2	4	3

bits =

6

error_status =

0

[coding_space, bits, error_status] = SGA_encoding('permutation',4,6,'a')

coding_space =

97	4	1	5	3	6
97	4	6	5	1	2
97	4	1	5	2	6
97	4	5	2	6	3

Example5 - messy encoding

[messy_coding_data_t, error_status] = SGA_encoding('messy',5,3,3)

messy_coding_data_t =

Columns 1 through 14

1	3	5	4	2	2	3	4	1	5	5	2	1	3
1	5	3	4	2	2	5	3	1	4	5	2	1	3
1	5	4	3	2	2	5	1	4	3	5	2	3	1

Column 15

4
4
4

error_status =

15

See Also

**SGA_decoding ,
SGA_selection ,
SGA_crossover,
SGA_mutation,
SGA_fitness_function,
SGA_fitness_evaluating,
SGA_benchmark_funcs,**

SGA_decoding

Decode data from coding space to real space

Syntax

```
[data_space,error_status ] = SGA_decoding(  
options, min_confines , max_confines , coding_space , bits )
```

Input:

options-- 'Binary' , 'b' : binary encoding method
 'Real' , 'r' : real number encoding method
 'Literal','Permutation','p','l' : literal permutation encoding method
 Permutation encoding for TSP
 Permutation encoding can be used in ordering
 problems, such as the traveling salesman problem
 or a task ordering problem. Every chromosome is
 a string of numbers, which represents number in a
 sequence. In the TSP each number would
 represent a city to be visited.

 'Gray' , 'g' : Gray encoding method
 'DNA' , 'd' : DNA encoding method
 'Messy' , 'm' : Messy encoding method

Output:

data_space -- is M*N matrix, where
 M is population
 N is number of variables,
 such as [x1,x2], N = 2

error_status = 0 -- ok
 1 -- fail

Description

in the method of 'X=Decimal*decimal_step+min_confines'
data_space is the decimal number which from coding_space
min_confines is the minimum of input value in decimal-space
max_confines is the maximum of input value in decimal-space

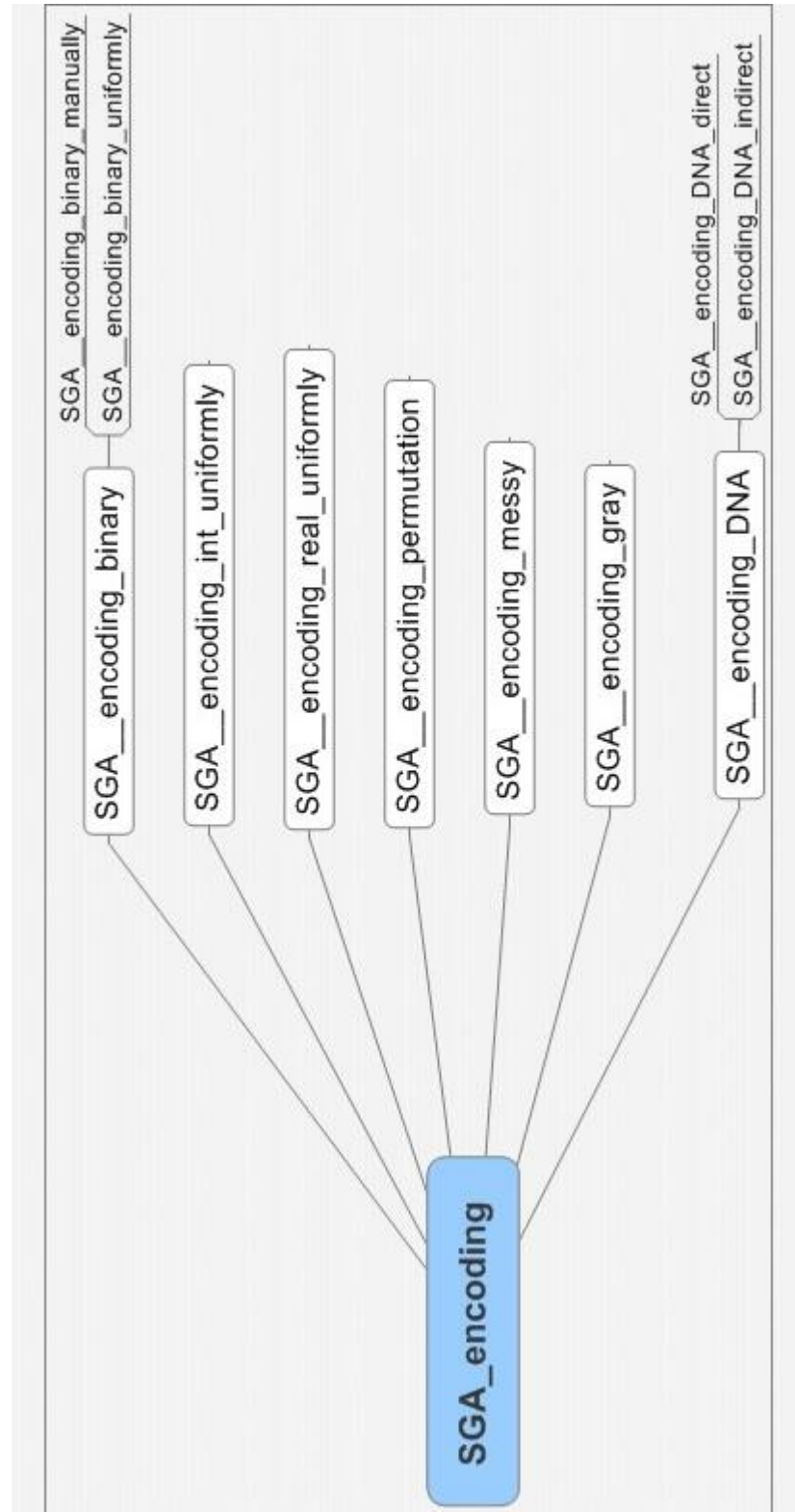
Examples

```
% f(x)=1./a+sin(b)+exp(c)+log2(d)+10 , a,b,c,d belong to [1,7],[2,8],[3,9],[4,10]  
[ coding_space , bits_sum , bits,istate ] = SGA__binary_encoding('b',  
                                          [1,2,3,4],[7,8,9,10],10,[0.01,0.01,0.01,0.01])
```

See Also

SGA_encoding ,
SGA_selection ,

SGA_crossover,
SGA_mutation,
SGA_fitness_function,
SGA_fitness_evaluating,
SGA_benchmark_funcs,



4.4.1 Binary En / Decoding

SGA__encoding_binary

Binary encoding

Syntax

[binary_pool, bits, error_status] =

SGA__encoding_binary(min_confines,max_confines,population,steps)

Input:

min_confines -- min confine for each variable
max_confines -- max confine for each variable
population -- population of individual

attention: when 'manual', design binary matrix as string format
such as: ['10010';'11111';'00000'](3*5)

data_space-- is original data matrix (m*n)

where: m -- population

n -- total length of all variables array

e.g.

	x1	x2	x3	x4	x...
data_space = [populaton1:	3.8	3.6	74	8	...
populaton2:	2	3	7.7	77	...
populaton3:	356	65	53	6	...

	x1	x2	x3	x4	x...
coding_space = [populaton1:	1010	0001	1010	0101	...
populaton2:	1000	0101	1110	1101	...
populaton3:	1011	0011	1011	1101	...

Output:

binary_pool-- is the coded space in 0 or 1 of binary format
bits -- bit lengths of each binary variable
error_status = 0 -- ok
 1 -- invalid parameter
 2 -- too many input arguments

Description

This function takes only 0 or 1 as element of matrix, it's so called 'binary'

Examples

[binary_pool, bits, error_status] = SGA__encoding_binary([1.1,1],[2,3],20,[0.01,0.01])

binary_pool =

Columns 1 through 12

1	1	0	1	0	1	0	1	0	0	0	0
1	0	0	1	0	1	1	0	1	1	1	1

1	0	1	1	1	0	1	1	0	1	0	1
0	1	0	1	0	1	1	1	0	0	0	0
1	1	0	1	0	1	1	0	0	0	1	0
0	0	0	0	1	1	1	0	1	0	1	0
0	1	1	0	1	1	0	1	1	1	1	0
0	0	1	1	1	0	1	1	0	0	0	1
1	1	0	1	1	1	1	1	0	1	1	1
0	1	0	0	0	1	0	0	1	0	0	0
1	0	0	0	0	0	1	1	1	1	1	0
1	1	1	0	0	1	0	1	1	1	1	1
0	1	0	0	1	0	1	0	1	1	1	1
0	1	0	1	0	1	0	1	1	1	0	1
0	0	1	1	1	0	1	0	1	1	1	1
1	1	0	0	0	1	0	0	1	0	0	1
0	1	0	1	1	0	1	0	1	0	0	1
0	0	1	0	1	0	0	1	1	1	1	0
1	1	0	0	0	0	0	0	0	0	0	1
0	1	0	0	1	0	0	0	1	1	0	0

Columns 13 through 15

1	0	1
1	0	0
0	1	0
0	1	1
1	0	1
1	0	1
0	0	0
1	0	0
0	1	0
1	0	1
1	1	0
1	0	0
1	1	0
0	0	0
0	0	1
1	1	0
0	1	0
0	1	1
0	0	0
0	1	0

bits =

7	8
---	---

error_status =

0

See Also

**SGA__encoding,
SGA__encoding_real_uniformly
SGA__encoding_permutation
SGA__encoding_gray,
SGA__encoding_int_uniformly ,
SGA__encoding_messy,
SGA__encoding_real_uniformly,
SGA__encoding_DNA_direct
SGA__encoding_DNA_indirect
SGA__encoding_binary_uniformly
SGA__encoding_binary_manually
SGA__encoding_binary**

SGA__encoding_binary_uniformly

Binary encoding in uniform distribution

Syntax

**[binary_pool ,error_status] = SGA__encoding_binary_uniformly(population ,
total_binary_length)**

Input:

**population -- population of each geration
total_binary_length -- length of chromosome in binary format**

Output:

binary_pool -- binary data matrix of population*total_binary_length

**error_status = 0 -- ok
 !0 -- fail**

Description

This function takes only 0 or 1 as element of matrix, it's so called 'binary'

Examples

[binary_pool ,error_status] = SGA__encoding_binary_uniformly(6 , 12)

binary_pool =

0	0	1	0	0	1	0	1	1	0	1	0
0	1	1	0	0	0	1	1	0	1	0	0
0	1	1	0	1	0	0	1	1	1	1	0
0	0	1	1	1	0	0	1	1	0	1	0
1	0	1	0	0	1	0	1	0	0	1	0

0 0 1 1 0 1 0 0 0 0 0 0

error_status =

0

See Also

SGA__encoding,
SGA__encoding_real_uniformly
SGA__encoding_permutation
SGA__encoding_gray,
SGA__encoding_int_uniformly ,
SGA__encoding_messy,
SGA__encoding_real_uniformly,
SGA__encoding_DNA_direct
SGA__encoding_DNA_indirect
SGA__encoding_binary_uniformly
SGA__encoding_binary_manually
SGA__encoding_binary

SGA__encoding_binary_manually

User defined binary encoding

Syntax

[binary_pool ,error_status] = SGA__encoding_binary_manually(population ,
total_binary_length, design_binary_matrix)

Input:

population -- population of each geration
total_binary_length -- length of chromosome in binary format
design_binary_matrix -- User defined binary matrix

Output:

binary_pool -- binary data matrix of population*total_binary_length
error_status = 0 -- ok
 !0 -- fail

Description

You can define your own binary strings here

Examples

[binary_pool ,error_status] = SGA__encoding_binary_manually(4 , 3 ,[1 0 0 ;1 1 0 ;1 1 1;
1 0 1])

binary_pool =

1	0	0
1	1	0
1	1	1
1	0	1

error_status =

0

See Also

SGA__encoding,
 SGA__encoding_real_uniformly
 SGA__encoding_permutation
 SGA__encoding_gray,
 SGA__encoding_int_uniformly ,
 SGA__encoding_messy,
 SGA__encoding_real_uniformly,
 SGA__encoding_DNA_direct
 SGA__encoding_DNA_indirect
 SGA__encoding_binary_uniformly
 SGA__encoding_binary_manually
 SGA__encoding_binary

SGA__decoding_binary

Binary Decoding

Syntax

[data_space, error_status] = SGA__decoding_binary(min_confines , max_confines , coding_space , bits)

Input:

min_confines-- is the minimum of input value in decimal-space
 max_confines-- is the maximum of input value in decimal-space
 coding_space-- is the coded space in 0 or 1
 bits-- is the length of every parameter

Output:

data_space-- is the search step in decimal space
 error_status-- = 0 -- ok
 !0-- fail

Description

Decoding is the second step in SGA(Simple Genetic Algorithm)
 this decoding function is decoding the binary value which coded by coding function ,that is to say you should use coding function first
 it can transfer binary value to decimal value

Examples

[coding_space,bits,error_status] =

```
SGA__binary_encoding( [1.1,1],[2,3],10,[0.01,0.01]);
```

```
[ data_space, error_status ] = SGA__decoding_binary([1.1,1],[2,3] , coding_space , bits)
```

See Also

```
SGA_decoding ,  
SGA__decoding_binary ,  
SGA__decoding_DNA_indirect,  
SGA__encoding_DNA_direct,  
SGA__decoding_gray,  
SGA__decoding_int,  
SGA__decoding_real,
```

4.4.2 Permutation En / Decoding

SGA__encoding_permutation

Permutation encoding

Syntax

```
[ coding_space,error_status ] =  
SGA__encoding_permutation( population , var_number )
```

```
[ coding_space,error_status ] =  
SGA__encoding_permutation( population , var_number , start_var_number )
```

Input:

```
population      -- population of individual  
var_number      -- total variable number  
start_var_number -- the start variable
```

Output:

```
coding_space      -- coding space  
error_status = 0  -- ok  
                 1 -- invalid parameter
```

Description

Permutation is to generate different elements for each string.

Examples

```
SGA__encoding_permutation( 5 , 10 )
```

ans =

```
10    8    7    2    4    3    1    5    9    6  
 3    5    6   10    4    2    7    9    1    8  
 4    1    3   10    9    8    2    6    7    5
```

2	9	8	5	1	7	10	3	6	4
6	4	3	8	1	10	5	7	9	2

SGA__encoding_permutation(5 , 10 , 1)

ans =

1	3	8	4	9	6	2	10	5	7
1	2	6	10	3	8	7	5	9	4
1	10	8	6	7	5	4	2	9	3
1	5	10	3	2	8	6	9	7	4
1	4	5	7	8	3	2	10	9	6

See Also

SGA__encoding,
 SGA__encoding_real_uniformly
 SGA__encoding_permutation
 SGA__encoding_gray,
 SGA__encoding_int_uniformly ,
 SGA__encoding_messy,
 SGA__encoding_real_uniformly,
 SGA__encoding_DNA_direct
 SGA__encoding_DNA_indirect
 SGA__encoding_binary_uniformly
 SGA__encoding_binary_manually
 SGA__encoding_binary

4.4.3 Gray En / Decoding

SGA__encoding_gray

gray encoding

Syntax

[gray_pool, bits, error_status] =...

SGA__encoding_gray(min_confines , max_confines , population , steps)

Input:

min_confines -- min confine for each variable
max_confines -- max confine for each variable
population -- population of individual
steps -- steps array for each variable

Output:

coding_space -- is the coded space in 0 or 1 of gray format

x1 x2 x3 x4 x...

coding_space = [populaton1: 1010|0001|1010|0101|...

```

populaton2: 1000|0101|1110|1101|...
populaton3: 1011|0011|1011|1101|...
bits          -- the length of encoded individuals
error_status = 0 -- ok
                1 -- invalid parameter
                2 -- too many input arguments

```

Description

Gray encoding is a certain kind of binary encoding method.

Examples

```

[ gray_pool, bits, error_status ] = SGA__encoding_gray( [1.1,1],[2,3],6,[0.01,0.01])
gray_pool =

```

Columns 1 through 12

0	1	0	0	0	1	1	0	0	1	0	1
1	0	1	1	1	0	1	1	1	0	1	0
0	1	0	0	0	1	1	1	0	0	0	0
1	0	0	1	0	1	0	0	1	0	1	0
0	0	1	0	0	0	0	0	0	1	1	1
1	0	1	0	0	0	0	0	0	0	0	0

Columns 13 through 15

1	1	1
0	1	0
1	0	0
1	1	1
0	1	0
1	1	1

bits =

7	8
---	---

error_status =

0

See Also

```

SGA__encoding,
SGA__encoding_real_uniformly
SGA__encoding_permutation
SGA__encoding_gray,
SGA__encoding_int_uniformly ,
SGA__encoding_messy,

```

SGA__encoding_real_uniformly,
SGA__encoding_DNA_direct
SGA__encoding_DNA_indirect
SGA__encoding_binary_uniformly
SGA__encoding_binary_manually
SGA__encoding_binary

SGA__decoding_gray

Gray decoding

Syntax

[decimal_pool, error_status] = SGA__decoding_gray(min_confines , max_confines , gray_pool , bits)

Input:

min_confines -- is the minimum of input value in decimal-space
max_confines -- is the maximum of input value in decimal-space
gray_pool -- is the coded space in 0 or 1
bits -- is the length of every parameter

Output:

decimal_pool -- is the search step in decimal space
error_status -- = 0 -- ok
 !0-- fail

Description

Examples

[gray_pool, bits, error_status] = SGA__encoding_gray([1.1,1],[2,3],6,[0.01,0.01]);
[decimal_pool, error_status] = SGA__decoding_gray([1.1,1] ,[2,3] , gray_pool , bits)

decimal_pool =

1.2559	2.7647
1.7520	2.7255
1.6102	2.2314
1.1425	2.5686
1.5677	1.7765
1.6528	2.6627

error_status =

See Also

SGA_decoding ,
 SGA__decoding_binary ,
 SGA__decoding_DNA_indirect,
 SGA__encoding_DNA_direct,
 SGA__decoding_gray,
 SGA__decoding_int,
 SGA__decoding_real,

4.4.4 Real&Integer En / Decoding**SGA__encoding_real_uniformly**

Real encoding

Syntax

```
[ real_pool,error_status ] = SGA__encoding_real_uniformly( min_confines , max_confines ,
                                                         population)
```

Input:

```

min_confines    -- min confine for each variable
max_confines    -- max confine for each variable
population      -- population of individual

```

Output:

```

coding_space      -- coding space
error_status =    0 -- ok
                  1 -- invalid parameter

```

Description**Examples**

```
[ coding_space,error_status ] = SGA__encoding_real_uniformly( [0,0],[2,3],10)
```

```
coding_space =
```

```

0.1054    1.1994
1.8836    1.5166
0.2999    0.5079
0.7687    1.5742
0.6221    1.9236
0.3371    0.0486
1.7933    2.5106
0.6454    2.4104
1.4680    2.0934

```


0.8218 1.3857

error_status =

0

See Also

SGA__encoding,
SGA__encoding_real_uniformly
SGA__encoding_permutation
SGA__encoding_gray,
SGA__encoding_int_uniformly ,
SGA__encoding_messy,
SGA__encoding_real_uniformly,
SGA__encoding_DNA_direct
SGA__encoding_DNA_indirect
SGA__encoding_binary_uniformly
SGA__encoding_binary_manually
SGA__encoding_binary

SGA__encoding_int_uniformly

Integer encoding

Syntax

```
[ int_pool ,error_status ] =
```

SGA__encoding_int_uniformly(min_confines , max_confines , population)

Input:

```
min_confines    -- min confine for each variable
max_confines    -- max confine for each variable
population      -- population of individual
```

Output:

```
coding_space -- the coded space , integer data matrix of
              population*total_binary_length
error_status = 0 -- ok
              !0 -- fail
```

Description

SGA__encoding_int_uniformly() is to generate data for Genetic Algorithms' operators where:
we use **randint()** to Generate matrix of uniformly distributed random integers

Examples

```
[ int_pool ,error_status ] = SGA__encoding_int_uniformly( [-10,0],[0,10],10 )
```

int_pool =

-10	0
-7	7
-2	4
-10	9
-9	4
-8	4
-9	8
-4	5
-8	2
-9	6

error_status =

0

See Also

**SGA__encoding,
SGA__encoding_real_uniformly
SGA__encoding_permutation
SGA__encoding_gray,
SGA__encoding_int_uniformly ,
SGA__encoding_messy,
SGA__encoding_real_uniformly,
SGA__encoding_DNA_direct
SGA__encoding_DNA_indirect
SGA__encoding_binary_uniformly
SGA__encoding_binary_manually
SGA__encoding_binary**

SGA__decoding_real

Binary decoding

Syntax

[data_space , error_status] = SGA__decoding_real(coding_space)

Input:

coding_space-- is the real coding data

Output:

**data_space -- is M*N matrix, where
M is population**

N is number of variables,
such as [x1,x2], N = 2

error_status = 0 -- ok
1 – fail

Description

By real decoding method, coding_space = data_space;

Examples

coding_space = [

2	6	5	1	3	4
4	6	2	5	3	1
4	3	6	1	5	2
1	6	5	2	4	3]

>> [data_space , error_status] = SGA__decoding_real(coding_space)

data_space =

2	6	5	1	3	4
4	6	2	5	3	1
4	3	6	1	5	2
1	6	5	2	4	3

error_status =

0

See Also

SGA_decoding ,
SGA__decoding_binary ,
SGA__decoding_DNA_indirect,
SGA__encoding_DNA_direct,
SGA__decoding_gray,
SGA__decoding_int,
SGA__decoding_real,

4.4.5 Messy En / Decoding

SGA__encoding_messy

Messy encoding

Syntax

[messy_coding_data_t, error_status] =
SGA__encoding_messy(population , element_num , group_num)

Input:

```

population      -- Population for GA
element_num    -- the element number in same group , e.g. n of (x1,x2,...,xn)
group_num      -- the number of groups in a chromosome ,
                   e.g., (group1)(group2)...(groupm)

```

Output:

messy_coding_data_t is a data structure as following:

```
messy_coding_data_t.messy_pool      = messy_pool;  
messy_coding_data_t.group_number   = group_num;  
messy_coding_data_t.element_number = element_num;
```

```
e.g.      group1  group2      groupm
messy_pool = [ populaton1: (2,2,1,4)(1,1,3,4)...(4,4,3,1)
                populaton2: (1,2,1,4)(2,2,2,2)...(4,4,1,1)
                populaton3: (3,4,1,1)(1,1,2,4)...(1,2,1,1)
```

```
error_status =    0 -- ok
                !0 -- fail
```

Description

Examples

```
[ messy_coding_data_t, error_status ] = SGA__encoding_messy( 5,3,3 )
```

messy_coding_data_t =

```

    messy_pool: [5x9 double]
    group_number: 3
    element_number: 3

```

error_status =

0

See Also

SGA__encoding,
SGA__encoding_real_uniformly
SGA__encoding_permutation
SGA__encoding_gray,
SGA__encoding_int_uniformly ,
SGA__encoding_messy,
SGA__encoding_real_uniformly,
SGA__encoding_DNA_direct
SGA__encoding_DNA_indirect
SGA__encoding_binary_uniformly

SGA__encoding_binary_manually
SGA__encoding_binary

4.4.6 DNA En / Decoding

SGA__encoding_DNA_direct

Direct DNA encoding

Syntax

[coding_space, error_status] =
SGA__encoding_DNA_direct(population , bits)

Input:

population -- chromosome populaiton
bits -- length of string

Output:

coding_space -- coding space
error_status = **0** -- ok
 1 -- invalid parameter
 2 -- too many input arguments

Description

Examples

[coding_space, error_status] = SGA__encoding_DNA_direct(4 , 16)

coding_space =

three_to_five: [4x16 char]
five_to_three: [4x16 char]

error_status =

0

See Also

SGA__encoding_gray ,
SGA__encoding_int_uniformly ,
SGA__encoding_messy,
SGA__encoding_real_uniformly,
SGA__encoding_DNA_indirect

SGA__encoding_DNA_indirect

In-direct DNA encoding

Syntax

```
[ coding_space, error_status ] =  
SGA__encoding_DNA_indirect( min_confines , max_confines , population , steps )
```

Input:

```
min_confines    -- min confine for each variable  
max_confines    -- max confine for each variable  
population      -- population of individual  
steps           -- steps for each individuals
```

Output:

```
coding_space      -- coding space  
error_status =    0 -- ok  
                  1 -- invalid parameter  
                  2 -- too many input arguments
```

Description

Examples

```
[ coding_space, error_status ] = SGA__encoding_DNA_indirect( [1,2],[3,4],6,0.01 )
```

```
coding_space =
```

```
three_to_five: [6x16 char]  
five_to_three: [6x16 char]
```

```
error_status =
```

```
0
```

See Also

```
SGA__encoding,  
SGA__encoding_real_uniformly  
SGA__encoding_permutation  
SGA__encoding_gray,  
SGA__encoding_int_uniformly ,  
SGA__encoding_messy,  
SGA__encoding_real_uniformly,  
SGA__encoding_DNA_direct  
SGA__encoding_DNA_indirect  
SGA__encoding_binary_uniformly  
SGA__encoding_binary_manually  
SGA__encoding_binary
```

```
SGA__encoding_DNA_generate_complementary_pair
```

To get DNA encoded string's complementary pair

Syntax

[complementary_pair, istate] =

SGA__encoding_DNA_generate_complementary_pair(pair)

Input:

pair -- DNA string

Output:

complementary_pair -- complementary pair of input DNA string

istate = 0 -- ok

1 -- invalid parameter

2 -- too many input arguments

Description

pair : 5' ATTAGCAT 3'

complementary_pair : 3' TAATCGTA 5';

3' <--> 5'

A <--> T

C <--> G

G <--> C

T <--> A

Examples

[coding_space, error_status] =

SGA__encoding_DNA_generate_complementary_pair('ATCG')

coding_space =

TAGC

error_status =

0

See Also

SGA__encoding,
SGA__encoding_real_uniformly
SGA__encoding_permutation
SGA__encoding_gray,
SGA__encoding_int_uniformly ,
SGA__encoding_messy,
SGA__encoding_real_uniformly,
SGA__encoding_DNA_direct
SGA__encoding_DNA_indirect
SGA__encoding_binary_uniformly

SGA__encoding_binary_manually
SGA__encoding_binary

SGA__decoding_int

Integer decoding

Syntax

[int_data , error_status] = SGA__decoding_int(int_pool)

Input:

int_pool-- is the real coding data

Output:

int_data -- is M*N matrix, where
M is population
N is number of variables,
such as [x1,x2], N = 2

error_status = 0 -- ok
1 -- fail

Description

Examples

int_pool = [1 3 4 6; 2 3 4 5; 11 22 3 4; 3 5 6 7];

[int_data , error_status] = SGA__decoding_int(int_pool)

int_data =

1	3	4	6
2	3	4	5
11	22	3	4
3	5	6	7

error_status =

0

See Also

SGA_decoding ,


```

SGA__decoding_binary ,
SGA__decoding_DNA_indirect,
SGA__encoding_DNA_direct,
SGA__decoding_gray,
SGA__decoding_int,
SGA__decoding_real,

```

4.5 Selection

4.5.1 Roulette Wheel Selection

SGA__selection_roulettewheel

Syntax

```

[ data_space , coding_space , istate ] = SGA__selection_roulettewheel( data_space ,
coding_space, fitness_value )

```

Description

Roulette Wheel Selection

The idea behind the roulette wheel selection technique is that each individual is given a chance to become a parent in proportion to its fitness. It is called roulette wheel selection as the chances of selecting a parent can be seen as spinning a roulette wheel with the size of the slot for each parent being proportional to its fitness. Obviously those with the largest fitness (slot sizes) have more chance of being chosen. Thus, it is possible for one member to dominate all the others and get selected a high proportion of the time.

Roulette wheel selection can be implemented as follows:

- 1.Sum the fitness of all the population members. Call this TF (total fitness).
- 2.Generate a random number n, between 0 and TF.
- 3.Return the first population member whose fitness added to the preceding population members is greater than or equal to n.

Examples

```

% fitness_value = [ 2.0 1.8 1.6 1.4 1.2 1.0 0.8 0.6 0.4 0.2 0.0 ];
% data_space = [ 11 22 33 44 55 66 77 88 99 1010 1111];
% coding_space = data_space;
% [ data_space , coding_space , istate ] = SGA__selection_roulettewheel( data_space ,
coding_space, fitness_value )
%
% data_space =
%
%      11
%      11
%      22
%      22
%      33

```

```

%      44
%      44
%      55
%      66
%      77
%      99
%
%
% coding_space =
%
%      11
%      11
%      22
%      22
%      33
%      44
%      44
%      55
%      66
%      77
%      99
%
%
% istate =
%
%      0

```

See Also

[SGA__selection_truncation](#)
[SGA__selection_stochastic](#)
[SGA__selection_truncation](#)
[SGA__selection_local](#)
[SGA_selection_properties](#)
[SGA_selection_pressure](#)

4.5.2 Stochastic Universal Selection

[SGA__selection_stochastic](#)

Syntax

```
[ data_space , coding_space , istate ] = SGA__selection_stochastic( data_space ,
coding_space, fitness_value )
```

Description

Stochastic universal sampling ensures a selection of offspring which is closer to what is deserved than roulette wheel selection.

Examples

```
% fitness_value = [ 2.0 1.8 1.6 1.4 1.2 1.0 0.8 0.6 0.4 0.2 0.0 ]';
% data_space = [ 11 22 33 44 55 66 77 88 99 1010 1111]';
% coding_space = data_space;
% [ data_space , coding_space , istate ] = SGA__selection_stochastic( data_space ,
coding_space, fitness_value )
%
% data_space =
%
%      11
%      11
%      22
%      22
%      33
%      44
%      44
%      55
%      66
%      77
%      99
%
%
% coding_space =
%
%      11
%      11
%      22
%      22
%      33
%      44
%      44
%      55
%      66
%      77
%      99
%
%
% istate =
%
%      0
```

See Also

[SGA__selection_roulettewheel](#)

[SGA__selection_truncation](#)

[SGA__selection_local](#)

SGA__selection_tournament
SGA_selection_properties
SGA_selection_pressure

4.5.3 Tournament Selection

SGA__selection_tournament

Syntax

[data_space , coding_space , istate] = SGA__selection_tournament(data_space , coding_space, fitness_value, tour_size)

Description

In tournament selection a number **Tour** of individuals is chosen randomly from the population and the best individual from this group is selected as parent. This process is repeated as often as individuals must be chosen. These selected parents produce uniform at random offspring. The parameter for tournament selection is the tournament size **Tour**. **Tour** takes values ranging from 2 to **Nind** (number of individuals in population).

Examples

```
% fitness_value = [ 2.0 1.8 1.6 1.4 1.2 1.0 0.8 0.6 0.4 0.2 0.0 ]';
% data_space = [ 11 22 33 44 55 66 77 88 99 1010 1111]';
% coding_space = data_space;
% tour_size = 5;
% [ data_space , coding_space , istate ] = SGA__selection_tournament( data_space ,
coding_space, fitness_value,tour_size )
%
% data_space =
%
%      55
%      55
%      55
%      55
%      22
%      22
%      22
%      22
%      22
%      55
%      22
%
%
% coding_space =
%
%      55
```

```

%      22
%      55
%      55
%      55
%      22
%      55
%      55
%      55
%      22
%      22
%
%
% istate =
%
%      0

```

See Also

[SGA__selection_roulettewheel](#)
[SGA__selection_stochastic](#)
[SGA__selection_tournament](#)
[SGA__selection_local](#)
[SGA_selection_properties](#)
[SGA_selection_pressure](#)

4.5.4 Truncation Selection

[SGA__selection_truncation](#)

Syntax

```
[ data_space , coding_space , istate ] = SGA__selection_truncation( data_space ,
coding_space, fitness_value, trunc_percent )
```

Description

Truncation selection :

Compared to the previous selection methods modeling natural selection truncation selection is an artificial selection method. It is used by breeders for LARGE populations/mass selection.

In truncation selection individuals are sorted according to their fitness. Only the best individuals are selected for parents. These selected parents produce uniform at random offspring. The parameter for truncation selection is the truncation threshold Trunc. Trunc indicates the proportion of the population to be selected as parents and takes values ranging from 50%-10%. Individuals below the truncation threshold do not produce offspring. The term selection intensity is often used in truncation selection.

Examples

```

% fitness_value = [ 2.0 1.8 1.6 1.4 1.2 1.0 0.8 0.6 0.4 0.2 0.0 ]';
% data_space = [ 11 22 33 44 55 66 77 88 99 1010 1111]';

```

```

% coding_space = data_space;
% trunc_percent = 0.5;
% [ data_space , coding_space , istate ] = SGA__selection_truncation( data_space ,
coding_space, fitness_value,trunc_percent )
% data_space =
%
%      44
%      33
%      44
%      66
%      11
%      66
%      22
%      55
%      66
%      44
%      55
%
%
% coding_space =
%
%      33
%      22
%      44
%      66
%      33
%      33
%      55
%      44
%      33
%      66
%      22
%
%
% istate =
%
%      0

```

See Also

SGA__selection_roulettewheel

SGA__selection_tournament

SGA__selection_stochastic

SGA__selection_local

4.5.5 Uniform Random Selection

SGA__selection_uniform_random

Syntax

[data_space , coding_space , istate] = SGA__selection_uniform_random(data_space , coding_space, fitness_value)

Description

Examples

```
% fitness_value = [ 2.0 1.8 1.6 1.4 1.2 1.0 0.8 0.6 0.4 0.2 0.0 ]';
% data_space = [ 11 22 33 44 55 66 77 88 99 1010 1111]';
% coding_space = data_space;
% [ data_space , coding_space , istate ] = SGA__selection_uniform_random( data_space ,
coding_space, fitness_value )
%
% data_space =
%
%      1111
%      33
%      77
%      66
%     1010
%      99
%      66
%      11
%     1010
%      55
%      77
%
%
% coding_space =
%
%      77
%      77
%      66
%      99
%      55
%     1010
%      99
%     1111
%      55
%     1010
%      66
%
%
%
% istate =
%
```

% 0

See Also

SGA__selection_roulettewheel

SGA__selection_tournament

SGA_selection_properties

SGA_selection_pressure

4.6 Crossover

4.6.1 binary single point crossover

SGA__crossover_binary_singlepoint

Syntax

```
[ decimal_space_crossover, binary_space_crossover,istate ] =  
SGA__binary_singlepoint_crossover( min_confines , max_confines ,  
decimal_space_selected , binary_space_selected , bits , probability_crossover )
```

Description

Examples

See Also

SGA__crossover_binary_singlepoint

SGA__crossover_real_int_singlepoint

4.6.2 real/integer single point crossover

SGA__crossover_real_int_singlepoint

Syntax

```
[ decimal_space,istate ] = SGA__real_int_singlepoint_crossover( decimal_space ,  
probability_crossover )
```

Description

Examples

```
% [ decimal_space_crossover ,istate ] =  
SGA__crossover_real_int_singlepoint( [1.232,2.34435,3.454;0.1 -0.342  
-343.3;-0.004,2.45,3.232;3 667 76] , 0.6 )  
%  
% decimal_space_crossover =  
%  
% 3.0000 667.0000 -343.3000  
% 0.1000 -0.3420 76.0000
```



```
% -0.0040    2.4500    3.2320
%  1.2320    2.3443    3.4540
%
%
% istate =
%
%      0
```

See Also

SGA__crossover_binary_singlepoint

SGA__crossover_real_int_singlepoint

4.6.3 TSP – CX

Syntax

```
[ path_code_string, istate ] = SGA__TSP_crossover_CX( path_code_string,
probability_crossover )
```

Description

Cycle Crossover (CX)

Examples

```
% [ path_code_string, istate ] = SGA__TSP_crossover_CX( [1 2 3 4 5 6 7 8 9;4 5 2 1 8 7 6
% 9 3;], .9 )
%
% path_code_string =
%
%      4      2      3      1      5      7      6      8      9
%      1      5      2      4      8      6      7      9      3
%
%
% istate =
%
%      0
```

See Also

SGA__TSP_crossover_BOOLMATRIX ,

SGA__TSP_crossover_CX ,

SGA__TSP_crossover_EAX,

SGA__TSP_crossover_OX,

SGA__TSP_crossover_PMX,

SGA__TSP_crossover_topological,

4.6.4 TSP – PMX

Syntax

```
[ path_code_string, istate ] = SGA__TSP_crossover_PMX( path_code_string,  
probability_crossover ,PMX_rate )
```

Description

Partially mapped Crossover (PMX) (1985) Goldberg and Lingle
Travel Salesman Problem--TSP Operator
Partially Matched Crossover, PMX

Examples

```
% [ path_code_string, istate ] = SGA__TSP_crossover_PMX( [1 2 3 4 5 6 7 8 9;4 5 2 1 8 7  
6 9 3;], .9 ,.9)  
%  
% path_code_string =  
%  
%      4      3      5      1      8      7      6      9      2  
%      1      8      3      4      9      6      7      2      5  
%  
%  
% istate =  
%  
%      0
```

See Also

SGA__TSP_crossover_BOOLMATRIX ,
SGA__TSP_crossover_CX ,
SGA__TSP_crossover_EAX,
SGA__TSP_crossover_OX,
SGA__TSP_crossover_PMX,
SGA__TSP_crossover_topological,

4.6.5 TSP – OX

Syntax

```
[ path_code_string, istate ] = SGA__TSP_crossover_OX( path_code_string,  
probability_crossover )
```

Description

Order Crossover (OX)

Examples

```
% [ path_code_string, istate ] = SGA__TSP_crossover_OX( [1 2 3 4 5 6 7 8 9;4 5 2 1 8 7 6  
9 3;], .9 )  
% path_code_string =  
%
```

```
% Columns 1 through 6
%
%      4      5      2      1      3      6
%      1      2      4      5      8      7
%
% Columns 7 through 9
%
%      7      8      9
%      6      9      3
```

See Also

```
SGA__TSP_crossover_BOOLMATRIX ,
SGA__TSP_crossover_CX ,
SGA__TSP_crossover_EAX,
SGA__TSP_crossover_OX,
SGA__TSP_crossover_PMX,
SGA__TSP_crossover_topological,
```

4.6.6 TSP – EAX

Syntax

```
[ path_code_string, istate ] = SGA__TSP_crossover_EAX( path_code_string,
probability_crossover , TSP_cost_matrix )
```

Description

Edge Assembly Crossover (EAX) (1997) Nagata and Kobayashi

Travel Salesman Problem--TSP Operator

Edge Assembly Crossover(EAX) is to do

Edge Recombination Crossover(ERX) With double edge marker,

Briefly:

$EAX = ERX + Edge_Marker$

there are two standard to carry out this ERX crossover

(1) Duplicated edge (Done)

(2) TSP cost matrix (Done)

Examples

```
% [ path_code_string, istate ] = SGA__TSP_crossover_EAX( [1 2 3 4 5 6 7 8 9;4 5 2 1 8 7
6 9 3;], .9 ,.9)
%
% path_code_string =
%
%      4      3      5      1      8      7      6      9      2
%      1      8      3      4      9      6      7      2      5
%
```

```
%
% istate =
%
%      0
```

See Also

```
SGA__TSP_crossover_BOOLMATRIX ,
SGA__TSP_crossover_CX ,
SGA__TSP_crossover_EAX,
SGA__TSP_crossover_OX,
SGA__TSP_crossover_PMX,
SGA__TSP_crossover_topological,
```

4.6.7 TSP – Topological

Syntax

Description

Examples

See Also

```
SGA__TSP_crossover_BOOLMATRIX ,
SGA__TSP_crossover_CX ,
SGA__TSP_crossover_EAX,
SGA__TSP_crossover_OX,
SGA__TSP_crossover_PMX,
SGA__TSP_crossover_topological,
```

4.6.8 TSP – BoolMatrix

Syntax

```
[ path_code_string, istate ] = SGA__TSP_crossover_BOOLMATRIX( path_code_string,
probability_crossover )
```

Description

Matrix Representations and Operators

Examples

```
% path_code_string = [ 1 2 3 4 5 6 7 8 9;
%                   4 1 2 8 7 6 9 3 5;
%                   2 1 4 5 6 3 7 8 9;
%                   9 6 4 5 7 2 1 3 1];
```

```

% probability_crossover = 1;
%
% [ path_code_string, istate ] = SGA__TSP_crossover_BOOLMATRIX( path_code_string,
probability_crossover )
% path_code_string =
%
%      1      2      3      4      5      6      7      8      9
%      4      1      2      8      7      6      9      3      5
%      1      2      3      4      5      6      7      8      9
%      1      2      3      4      5      6      7      8      9
%
%
% istate =
%
%      0

```

See Also

```

SGA__TSP_crossover_BOOLMATRIX ,
SGA__TSP_crossover_CX ,
SGA__TSP_crossover_EAX,
SGA__TSP_crossover_OX,
SGA__TSP_crossover_PMX,
SGA__TSP_crossover_topological,

```

4.7 Mutation

4.7.1 binary single point mutation

Syntax

Description

Examples

```
0
```

See Also

4.7.2 real/integer single point mutation

Syntax

Description

Examples

0

See Also

4.7.3 TSP – Displacement

Syntax

```
[ path_code_string, istate ] = SGA__TSP_mutation_displacement( path_code_string,  
probability_mutation )
```

Description

Displacement Mutation (DM)

Examples

```
% [ path_code_string, istate ] = SGA__TSP_mutation_displacement( [1 2 3 4 5; 11 22 33 44  
55], 1 )  
%  
% path_code_string =  
%  
%      3      1      2      4      5  
%     11     22     33     44     55
```

See Also

SGA__TSP_mutation_displacement ,
SGA__TSP_mutation_insertion ,
SGA__TSP_mutation_inversion,
SGA__TSP_mutation_reciprocalexchange,

4.7.4 TSP – Insertion

Syntax

```
[ path_code_string, istate ] = SGA__TSP_mutation_insertion( path_code_string,  
probability_mutation )
```

Description

Inversion Mutation for TSP problem:

Select two points along the permutation,

cut it at these points and re-insert the reversed string.

(1 2 | 3 4 5 6 | 7 8 9) --> (1 2 | 6 5 4 3 | 7 8 9).

Examples

```
% [ path_code_string, istate ] = SGA__TSP_mutation_insertion( [10 20 30 40 50 60 70; 11
22 33 44 55 66 77], 1 )
%
% path_code_string =
%
%      10      20      30      40      50      60      70
%      44      33      22      11      55      66      77
%
%
% istate =
%
%      0
```

See Also

SGA__TSP_mutation_displacement ,
SGA__TSP_mutation_insertion ,
SGA__TSP_mutation_inversion,
SGA__TSP_mutation_reciprocalexchange,

4.7.5 TSP – Inversion

Syntax

```
[ path_code_string, istate ] = SGA__TSP_mutation_inversion( path_code_string,
probability_mutation )
```

Description

Inversion Mutation for TSP problem:

Select two points along the permutation,
cut it at these points and re-insert the reversed string.
(1 2 | 3 4 5 6 | 7 8 9) --> (1 2 | 6 5 4 3 | 7 8 9).

Examples

```
% [ path_code_string, istate ] = SGA__TSP_mutation_inversion( [10 20 30 40 50 60 70; 11
22 33 44 55 66 77], 1 )
%
% path_code_string =
%
%      10      20      30      40      50      60      70
%      44      33      22      11      55      66      77
%
%
```

```
% istate =
%
%      0
```

See Also

SGA__TSP_mutation_displacement ,
 SGA__TSP_mutation_insertion ,
 SGA__TSP_mutation_inversion,
 SGA__TSP_mutation_reciprocalexchange,

4.7.6 TSP – Reciprocal Exchange

Syntax

```
[ path_code_string, istate ] = SGA__TSP_mutation_reciprocalexchange( path_code_string,
probability_mutation )
```

Description

Reciprocal Exchange: Swap two cities.

Examples

```
% [ path_code_string, istate ] = SGA__TSP_mutation_reciprocalexchange( [1 2 3 4 5; 11 22
33 44 55], 1 )
%
% path_code_string =
%
%      1      5      3      4      2
%     11     22     44     33     55
%
%
% istate =
%
%      0
```

See Also

SGA__TSP_mutation_displacement ,
 SGA__TSP_mutation_insertion ,
 SGA__TSP_mutation_inversion,
 SGA__TSP_mutation_reciprocalexchange,

4.8 Fitness Function Design

4.8.1 overview

4.8.2 fitness scaling /sharing/windowing

4.8.3 Math Function as Fitness Function

4.8.4 Table Data as Fitness Function

4.8.5 Constraint function design

4.8.6 Bench functions design

4.9 TSP

4.9.1 TSP path generation

Syntax

`[path_code_string,error_status] = SGA__TSP_path_generation(population ,
city_number)`

Description

to initialize tsp city list

travelling path begin from City 1,

and end By City city_number : [1 , N]

Examples

```
% SGA__TSP_path_generation( 5 , 3 )  
%  
% ans =  
%  
%      3      1      2  
%      1      2      3  
%      1      2      3  
%      3      2      1  
%      3      2      1
```

See Also

`SGA__TSP_path_generation ,`

`SGA__TSP_set_NaN_to_cost_matrix ,`

`SGA__TSP_xypoint_to_costmatrix,`

4.9.2 TSP cost evaluation

Syntax

`[ matrix_path_cost ] = SGA__TSP_cost_evaluating( path_matrix, TSP_cost_matrix )`

Description

to evaluate the cost for TSP

Examples

See SGALAB_demo_TSP_13cities.m

See Also

SGA__TSP_FITNESS_function ,

SGA__TSP_cost_evaluating ,

4.9.3 TSP fitness function

Syntax

[fitness] = SGA__TSP_FITNESS_function(path_array , TSP_cost_matrix)

Description

to define the fitness function for TSP

Examples

See SGALAB_demo_TSP_13cities.m

See Also

SGA__TSP_FITNESS_function ,

SGA__TSP_cost_evaluating ,

4.10 Multi-objective Genetic Algorithms

4.10.1 Sum of Weighted Objectives (Non-Pareto)

Syntax

Description

Examples

See Also

4.10.2 Goal Programming (Non-Pareto)

Syntax

Description

Examples

See Also

4.10.3 Epsilon Constraint (Non-Pareto)

Syntax

Description

Examples

See Also

4.10.4 VEGA (Non-Pareto)

SGA_entry_MO_NON_Pareto_VEGA

Syntax

```
[ maxfitness , minfitness , meanfitness , now_generation ,  
now_probability_crossover,best_decimal_space , best_coding_space ,error_status] =  
SGA_entry_MO_NON_Pareto_VEGA( options,min_confines,max_confines,probability_cross  
over,probability_mutation,population,decimal_step,max_generation,convergence_method,  
max_no_change_probability_crossover_generation,deta_fitness_max,max_probability_cro  
ssover,max_no_change_fitness_generation,probability_crossover_step)
```

Description

VEGA = Vector Evaluated Genetic Algorithm

Examples

See SGALAB_demo_MO_VEGA.m

See Also

SGA_entry_MO_showall

SGA_entry_MO_Pareto_MOGA

SGA_entry_MO_Pareto_NPGA

SGA_entry_MO_Pareto_NSGA

SGA_entry_MO_Pareto_NSGAII

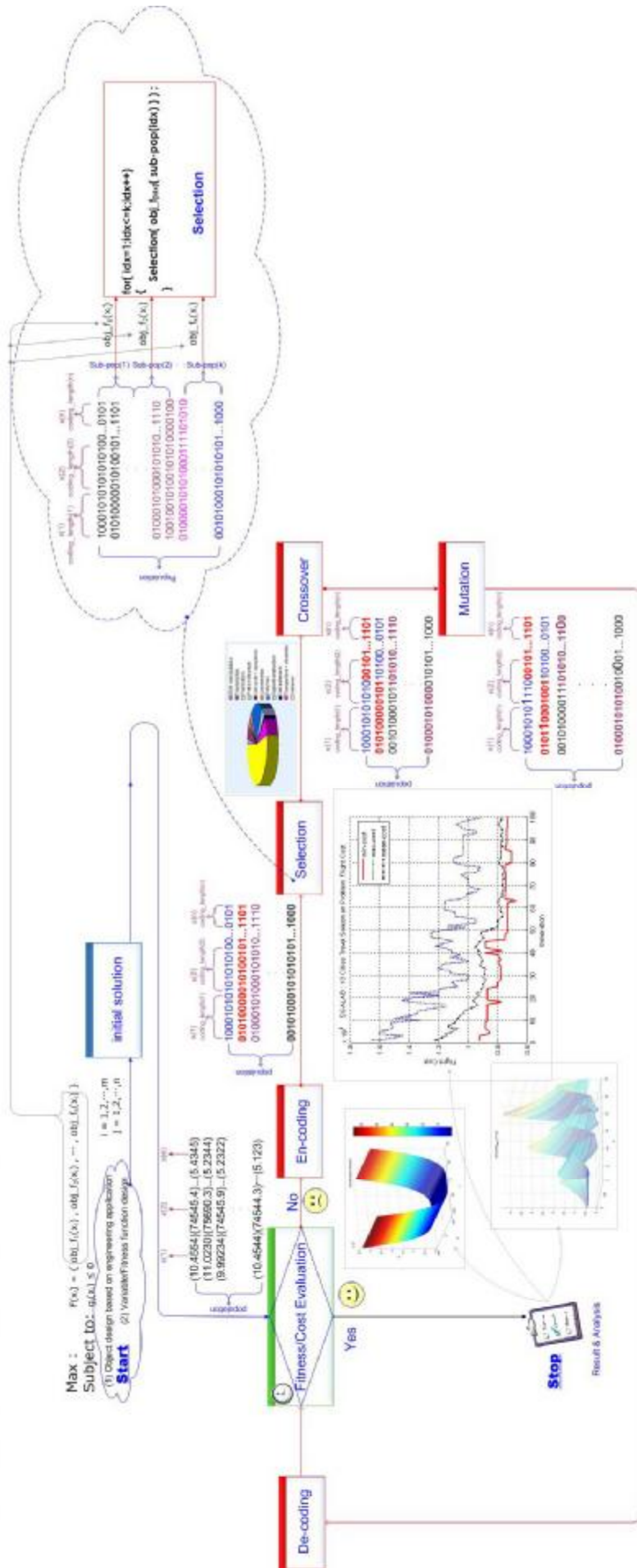
SGA_entry_MO_Pareto_PAES

SGA_entry_MO_Pareto_SPDE
SGA_entry_MO_Pareto_SPEA
SGA_entry_MO_Pareto_SPEA2
SGA_entry_MO_NON_Pareto_VEGA
SGA_entry_MO_Goal_Attainment_fitness

Schaffer - Vector Evaluated Genetic Algorithm(VEGA)

J. David Schaffer, Multiple objective optimization with vector evaluated genetic algorithms. In: Genetic Algorithms and their Applications: Proceedings of the First International Conference on Genetic Algorithms, pages 95-100, Hillsdale, NJ, 1985. Lawrence Erlbaum

Chen Yi
chenyic@foxmail.com
20th Nov 2025



4.10.5 MOGA (Pareto)

Syntax

```
[ maxfitness , minfitness , meanfitness , now_generation ,  
now_probability_crossover,best_decimal_space , best_coding_space ,error_status ] =  
SGA_entry_MO_Pareto_MOGA( options,min_confines,max_confines,probability_crossover,  
probability_mutation,population,decimal_step,max_generation,convergence_method,max  
_no_change_probability_crossover_generation,deta_fitness_max,max_probability_crossov  
er,max_no_change_fitness_generation,probability_crossover_step)
```

Description

MOGA method

Examples

See SGALAB_demo_MO_MOGA.m

See Also

SGA_entry_MO_showall
SGA_entry_MO_Pareto_MOGA
SGA_entry_MO_Pareto_NPGA
SGA_entry_MO_Pareto_NSGA
SGA_entry_MO_Pareto_NSGAII
SGA_entry_MO_Pareto_PAES
SGA_entry_MO_Pareto_SPDE
SGA_entry_MO_Pareto_SPEA
SGA_entry_MO_Pareto_SPEA2
SGA_entry_MO_NON_Pareto_VEGA
SGA_entry_MO_Goal_Attainment_fitness

Multiobjective Genetic Algorithm (MOGA)

Fonseca, C. M. & P. J. Fleming: "Genetic algorithms for multiobjective optimization: formulation, discussion and generalization", *Proc. of the 5th Inter. Conf. on GAs*, pp. 416-423, 1993

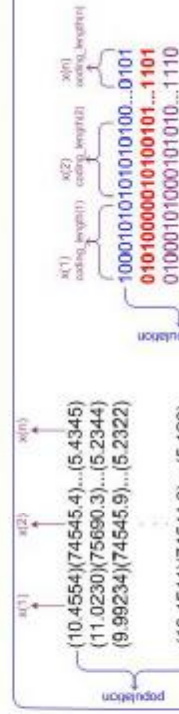
Chen Yi
chenyi2005@gmail.com
28th Nov. 2005

Max : $F(x) = \{ \text{obj}_1(x), \text{obj}_2(x), \dots, \text{obj}_m(x) \}$

Subject to: $g_i(x) \leq 0$
 $i = 1, 2, \dots, m$
 $j = 1, 2, \dots, n$

(1) Object design based on engineering application
Start (2) Variable/Fitness function design

initial solution



De-coding

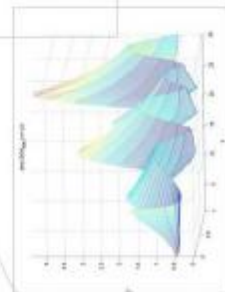
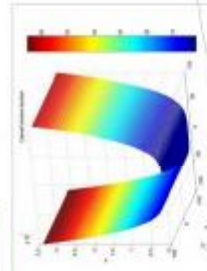
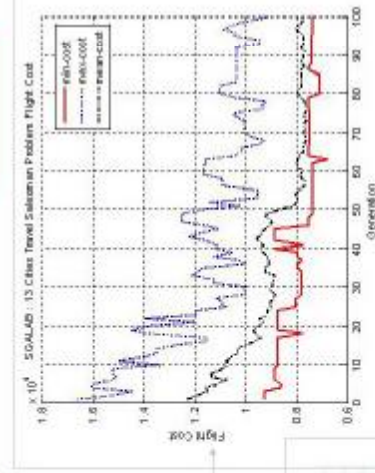
En-coding

MOGA Block

Selection

Crossover

Mutation



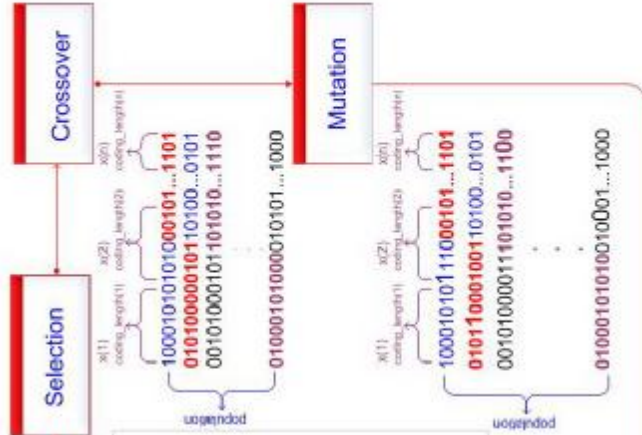
Stop

Result & Analysis

Multiobjective Genetic Algorithm (MOGA: Fonseca & Fleming 1993)

1. **Fitness Pareto ranking** -- for each chromosome c calculate the number of chromosomes that dominate it (n). Let rank $ri=1+ni$;
2. **Fitness assignment** -- Assign fitness to c using linear interpolation so that c with lowest rank has maximum fitness;
3. Fitness **Average** fitnesses of chromosomes with same rank and assign as the new fitness;
4. **Fitness Sharing** -- Use niching methods to distribute chromosomes over the Pareto-optimal region.

The rest of the algorithm is a standard GA.



4.10.6 NPGA (Pareto)

Syntax

```
[ maxfitness , minfitness , meanfitness , now_generation ,  
now_probability_crossover,best_decimal_space , best_coding_space ,error_status] =  
SGA_entry_MO_Pareto_NPGA( options,min_confines,max_confines,probability_crossover,p  
robability_mutation,population,decimal_step,max_generation,convergence_method,max_  
no_change_probability_crossover_generation,deta_fitness_max,max_probability_crossove  
r,max_no_change_fitness_generation,probability_crossover_step)
```

Description

NPGA method

Examples

See SGALAB_demo_MO_NPGA.m

See Also

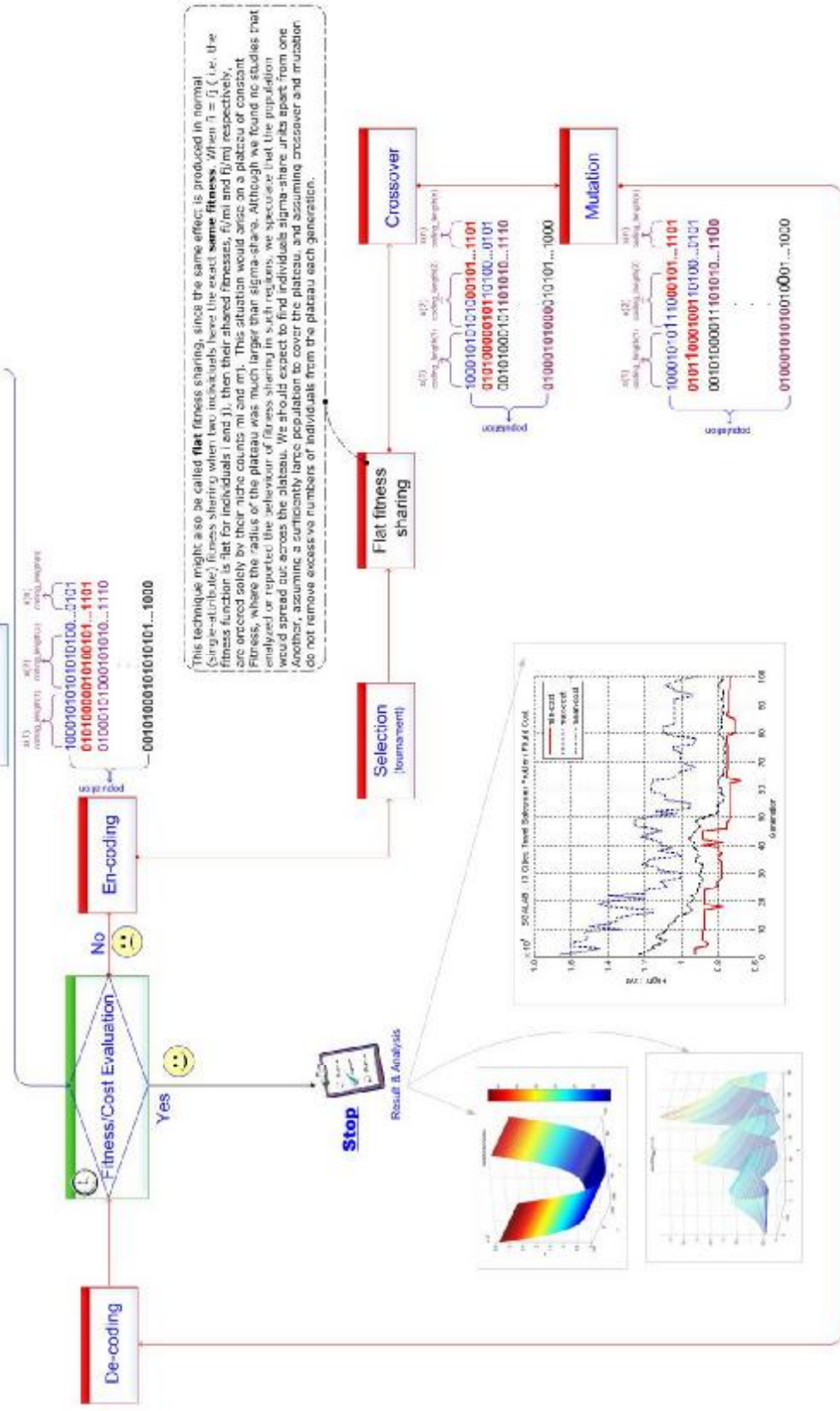
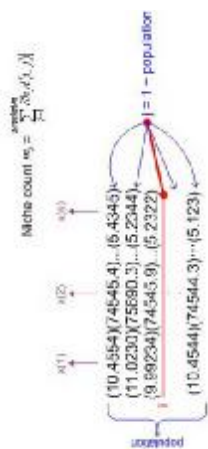
SGA_entry_MO_showall
SGA_entry_MO_Pareto_MOGA
SGA_entry_MO_Pareto_NPGA
SGA_entry_MO_Pareto_NSGA
SGA_entry_MO_Pareto_NSGAII
SGA_entry_MO_Pareto_PAES
SGA_entry_MO_Pareto_SPDE
SGA_entry_MO_Pareto_SPEA
SGA_entry_MO_Pareto_SPEA2
SGA_entry_MO_NON_Pareto_VEGA
SGA_entry_MO_Goal_Attainment_fitness

Niched Pareto Genetic Algorithm (NPGA)

Hern, J. and N. Nafpliotis (1993), Multiobjective optimization using the niched pareto genetic algorithm, IJGAL, Report 93005, Illinois Genetic Algorithms Laboratory, University of Illinois, Urbana, Champaign.

Max : $F(x) = \{obj_1(x), obj_2(x), \dots, obj_L(x)\}$
 Subject to: $-g_i(x) \leq 0$
 (1) Object design based on engineering application
 (2) Variable fitness function design
 Start $i = 1, 2, \dots, m$
 $j = 1, 2, \dots, n$

Chen Yi
 chen2005@gmail.com
 28th Nov, 2015



This technique might also be called flat fitness sharing, since the same effect is produced in normal (Shapiro-style) fitness sharing when two individuals have the exact same fitness. When $n = 1$ (i.e. the fitness function is flat for individuals i and j), then their shared fitnesses $f_{\text{share}}(i)$ and $f_{\text{share}}(j)$ respectively are averaged solely by their niche counts n_i and n_j . This situation would arise on a plateau of constant fitness, where the radius of the plateau was much larger than sigma-share. Although we found no studies that analyzed or reported the behavior of fitness sharing in such cases, we speculate that the population would spread out across the plateau. We should expect to find individuals sigma-share units apart from one another, assuming a sufficiently large population to cover the plateau, and assuming crossover and mutation do not remove excess individuals from the plateau each generation.

4.10.7 NSGA (Pareto)

Syntax

```
[ maxfitness , minfitness , meanfitness , now_generation ,  
now_probability_crossover,best_decimal_space , best_coding_space ,error_status] =  
SGA_entry_MO_Pareto_NSGA( options,min_confines,max_confines,probability_crossover,  
probability_mutation,population,decimal_step,max_generation,convergence_method,max  
_no_change_probability_crossover_generation,deta_fitness_max,max_probability_crossov  
er,max_no_change_fitness_generation,probability_crossover_step)
```

Description

NSGA Method

Examples

See SGALAB_demo_MO_NSGA.m

See Also

SGA_entry_MO_showall

SGA_entry_MO_Pareto_MOGA

SGA_entry_MO_Pareto_NPGA

SGA_entry_MO_Pareto_NSGA

SGA_entry_MO_Pareto_NSGAII

SGA_entry_MO_Pareto_PAES

SGA_entry_MO_Pareto_SPDE

SGA_entry_MO_Pareto_SPEA

SGA_entry_MO_Pareto_SPEA2

SGA_entry_MO_NON_Pareto_VEGA

SGA_entry_MO_Goal_Attainment_fitness

Non-dominated Sorting Genetic Algorithm(NSGA)

Srinivas, N. and K. Deb (1994), Multiobjective optimization using nondominated sorting in genetic algorithms, *Evolutionary Computation*, 2(3), 221-248.

Chen Yi
chenyic00@gmail.com
20th Nov, 2008

Max : $F(x) = (obj_1(x), obj_2(x), \dots, obj_k(x))$
Subject to: $g_i(x) \leq 0$
Start (1) Object design based on engineering application
(2) Variable fitness function design
 $i = 1, 2, \dots, m$
 $j = 1, 2, \dots, n$

Initial solution
Population
(10,4554)(7,6545,4)...(5,4345)
(11,0230)(7,5690,3)...(5,2344)
(8,99234)(7,6545,9)...(5,2322)
(10,4544)(7,4544,3)...(5,123)

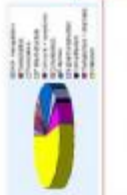
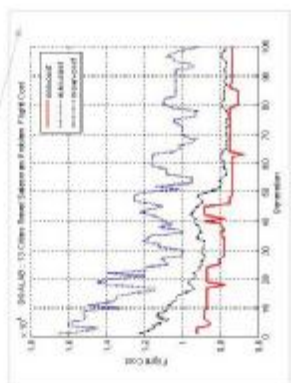
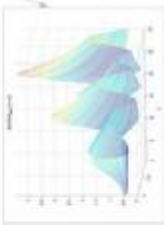
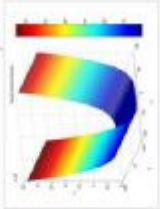
En-coding
010010101010100...0101
01010000010100101...1101
01000101000101010...1110
00101000101010101...1000

Fitness/Cost Evaluation
Yes
No

De-coding

Pareto-ranking

Stop
Result & Analysis



Crossover

Population
010010101000101...1000
01010000010100101...1101
01000101000101010...1110

Mutation

Population
010010101000101...1000
01010000010100101...1101
01000101000101010...1110

Selection
stochastic selection
proportional selection

NSGA Block

Goldberg's method
Input : chromosomes $x(i)$, $i = 1, 2, \dots, \text{population}$
Output : rank of fitness
Step 1: rank 1 to the non-dominated chromosomes;
Step 2: removing ranked chromosomes for input;
Step 3: go on next rank, rank 2, until all the fitness are ranked;
Step 4: until all the fitness are ranked.

MOGA method(MOGA-Pareto Sort Filtering, 1993)
Input : chromosomes $x(i)$, $i = 1, 2, \dots, \text{population}$
Output : rank of fitness
Step 1: rank 1 is given to the non-dominated fitness values;
Step 2: removing them from container;
Step 3: finding the next non-dominated fitness values;
Step 4: process continued until the 2nd & 3rd rank on E ranked.

Non-dominated Sorting GA (NSGA Deb, 1995)
1. Let : chromosomes $x(i)$, $i = 1, 2, \dots, \text{population}$
Output : fitness
Step 1: The non-dominated individuals are first identified and assigned a fitness value;
Step 2: To maintain diversity in the population, these individuals are shared with their dummy fitness value;
Step 3: after sharing, these non-dominated individuals are grouped together;
Step 4: The second non-dominated front is identified, if it is separated or dominated are assigned a dummy fitness value;
Step 5: after that, the 3rd, 4th, 5th, 6th, 7th, 8th, 9th, 10th, 11th, 12th, 13th, 14th, 15th, 16th, 17th, 18th, 19th, 20th, 21st, 22nd, 23rd, 24th, 25th, 26th, 27th, 28th, 29th, 30th, 31st, 32nd, 33rd, 34th, 35th, 36th, 37th, 38th, 39th, 40th, 41st, 42nd, 43rd, 44th, 45th, 46th, 47th, 48th, 49th, 50th, 51st, 52nd, 53rd, 54th, 55th, 56th, 57th, 58th, 59th, 60th, 61st, 62nd, 63rd, 64th, 65th, 66th, 67th, 68th, 69th, 70th, 71st, 72nd, 73rd, 74th, 75th, 76th, 77th, 78th, 79th, 80th, 81st, 82nd, 83rd, 84th, 85th, 86th, 87th, 88th, 89th, 90th, 91st, 92nd, 93rd, 94th, 95th, 96th, 97th, 98th, 99th, 100th, 101st, 102nd, 103rd, 104th, 105th, 106th, 107th, 108th, 109th, 110th, 111st, 112nd, 113th, 114th, 115th, 116th, 117th, 118th, 119th, 120th, 121st, 122nd, 123rd, 124th, 125th, 126th, 127th, 128th, 129th, 130th, 131st, 132nd, 133rd, 134th, 135th, 136th, 137th, 138th, 139th, 140th, 141st, 142nd, 143rd, 144th, 145th, 146th, 147th, 148th, 149th, 150th, 151st, 152nd, 153rd, 154th, 155th, 156th, 157th, 158th, 159th, 160th, 161st, 162nd, 163rd, 164th, 165th, 166th, 167th, 168th, 169th, 170th, 171st, 172nd, 173rd, 174th, 175th, 176th, 177th, 178th, 179th, 180th, 181st, 182nd, 183rd, 184th, 185th, 186th, 187th, 188th, 189th, 190th, 191st, 192nd, 193rd, 194th, 195th, 196th, 197th, 198th, 199th, 200th, 201st, 202nd, 203rd, 204th, 205th, 206th, 207th, 208th, 209th, 210th, 211st, 212nd, 213rd, 214th, 215th, 216th, 217th, 218th, 219th, 220th, 221st, 222nd, 223rd, 224th, 225th, 226th, 227th, 228th, 229th, 230th, 231st, 232nd, 233rd, 234th, 235th, 236th, 237th, 238th, 239th, 240th, 241st, 242nd, 243rd, 244th, 245th, 246th, 247th, 248th, 249th, 250th, 251st, 252nd, 253rd, 254th, 255th, 256th, 257th, 258th, 259th, 260th, 261st, 262nd, 263rd, 264th, 265th, 266th, 267th, 268th, 269th, 270th, 271st, 272nd, 273rd, 274th, 275th, 276th, 277th, 278th, 279th, 280th, 281st, 282nd, 283rd, 284th, 285th, 286th, 287th, 288th, 289th, 290th, 291st, 292nd, 293rd, 294th, 295th, 296th, 297th, 298th, 299th, 300th, 301st, 302nd, 303rd, 304th, 305th, 306th, 307th, 308th, 309th, 310th, 311st, 312nd, 313rd, 314th, 315th, 316th, 317th, 318th, 319th, 320th, 321st, 322nd, 323rd, 324th, 325th, 326th, 327th, 328th, 329th, 330th, 331st, 332nd, 333rd, 334th, 335th, 336th, 337th, 338th, 339th, 340th, 341st, 342nd, 343rd, 344th, 345th, 346th, 347th, 348th, 349th, 350th, 351st, 352nd, 353rd, 354th, 355th, 356th, 357th, 358th, 359th, 360th, 361st, 362nd, 363rd, 364th, 365th, 366th, 367th, 368th, 369th, 370th, 371st, 372nd, 373rd, 374th, 375th, 376th, 377th, 378th, 379th, 380th, 381st, 382nd, 383rd, 384th, 385th, 386th, 387th, 388th, 389th, 390th, 391st, 392nd, 393rd, 394th, 395th, 396th, 397th, 398th, 399th, 400th, 401st, 402nd, 403rd, 404th, 405th, 406th, 407th, 408th, 409th, 410th, 411st, 412nd, 413rd, 414th, 415th, 416th, 417th, 418th, 419th, 420th, 421st, 422nd, 423rd, 424th, 425th, 426th, 427th, 428th, 429th, 430th, 431st, 432nd, 433rd, 434th, 435th, 436th, 437th, 438th, 439th, 440th, 441st, 442nd, 443rd, 444th, 445th, 446th, 447th, 448th, 449th, 450th, 451st, 452nd, 453rd, 454th, 455th, 456th, 457th, 458th, 459th, 460th, 461st, 462nd, 463rd, 464th, 465th, 466th, 467th, 468th, 469th, 470th, 471st, 472nd, 473rd, 474th, 475th, 476th, 477th, 478th, 479th, 480th, 481st, 482nd, 483rd, 484th, 485th, 486th, 487th, 488th, 489th, 490th, 491st, 492nd, 493rd, 494th, 495th, 496th, 497th, 498th, 499th, 500th, 501st, 502nd, 503rd, 504th, 505th, 506th, 507th, 508th, 509th, 510th, 511st, 512nd, 513rd, 514th, 515th, 516th, 517th, 518th, 519th, 520th, 521st, 522nd, 523rd, 524th, 525th, 526th, 527th, 528th, 529th, 530th, 531st, 532nd, 533rd, 534th, 535th, 536th, 537th, 538th, 539th, 540th, 541st, 542nd, 543rd, 544th, 545th, 546th, 547th, 548th, 549th, 550th, 551st, 552nd, 553rd, 554th, 555th, 556th, 557th, 558th, 559th, 560th, 561st, 562nd, 563rd, 564th, 565th, 566th, 567th, 568th, 569th, 570th, 571st, 572nd, 573rd, 574th, 575th, 576th, 577th, 578th, 579th, 580th, 581st, 582nd, 583rd, 584th, 585th, 586th, 587th, 588th, 589th, 590th, 591st, 592nd, 593rd, 594th, 595th, 596th, 597th, 598th, 599th, 600th, 601st, 602nd, 603rd, 604th, 605th, 606th, 607th, 608th, 609th, 610th, 611st, 612nd, 613rd, 614th, 615th, 616th, 617th, 618th, 619th, 620th, 621st, 622nd, 623rd, 624th, 625th, 626th, 627th, 628th, 629th, 630th, 631st, 632nd, 633rd, 634th, 635th, 636th, 637th, 638th, 639th, 640th, 641st, 642nd, 643rd, 644th, 645th, 646th, 647th, 648th, 649th, 650th, 651st, 652nd, 653rd, 654th, 655th, 656th, 657th, 658th, 659th, 660th, 661st, 662nd, 663rd, 664th, 665th, 666th, 667th, 668th, 669th, 670th, 671st, 672nd, 673rd, 674th, 675th, 676th, 677th, 678th, 679th, 680th, 681st, 682nd, 683rd, 684th, 685th, 686th, 687th, 688th, 689th, 690th, 691st, 692nd, 693rd, 694th, 695th, 696th, 697th, 698th, 699th, 700th, 701st, 702nd, 703rd, 704th, 705th, 706th, 707th, 708th, 709th, 710th, 711st, 712nd, 713rd, 714th, 715th, 716th, 717th, 718th, 719th, 720th, 721st, 722nd, 723rd, 724th, 725th, 726th, 727th, 728th, 729th, 730th, 731st, 732nd, 733rd, 734th, 735th, 736th, 737th, 738th, 739th, 740th, 741st, 742nd, 743rd, 744th, 745th, 746th, 747th, 748th, 749th, 750th, 751st, 752nd, 753rd, 754th, 755th, 756th, 757th, 758th, 759th, 760th, 761st, 762nd, 763rd, 764th, 765th, 766th, 767th, 768th, 769th, 770th, 771st, 772nd, 773rd, 774th, 775th, 776th, 777th, 778th, 779th, 780th, 781st, 782nd, 783rd, 784th, 785th, 786th, 787th, 788th, 789th, 790th, 791st, 792nd, 793rd, 794th, 795th, 796th, 797th, 798th, 799th, 800th, 801st, 802nd, 803rd, 804th, 805th, 806th, 807th, 808th, 809th, 810th, 811st, 812nd, 813rd, 814th, 815th, 816th, 817th, 818th, 819th, 820th, 821st, 822nd, 823rd, 824th, 825th, 826th, 827th, 828th, 829th, 830th, 831st, 832nd, 833rd, 834th, 835th, 836th, 837th, 838th, 839th, 840th, 841st, 842nd, 843rd, 844th, 845th, 846th, 847th, 848th, 849th, 850th, 851st, 852nd, 853rd, 854th, 855th, 856th, 857th, 858th, 859th, 860th, 861st, 862nd, 863rd, 864th, 865th, 866th, 867th, 868th, 869th, 870th, 871st, 872nd, 873rd, 874th, 875th, 876th, 877th, 878th, 879th, 880th, 881st, 882nd, 883rd, 884th, 885th, 886th, 887th, 888th, 889th, 890th, 891st, 892nd, 893rd, 894th, 895th, 896th, 897th, 898th, 899th, 900th, 901st, 902nd, 903rd, 904th, 905th, 906th, 907th, 908th, 909th, 910th, 911st, 912nd, 913rd, 914th, 915th, 916th, 917th, 918th, 919th, 920th, 921st, 922nd, 923rd, 924th, 925th, 926th, 927th, 928th, 929th, 930th, 931st, 932nd, 933rd, 934th, 935th, 936th, 937th, 938th, 939th, 940th, 941st, 942nd, 943rd, 944th, 945th, 946th, 947th, 948th, 949th, 950th, 951st, 952nd, 953rd, 954th, 955th, 956th, 957th, 958th, 959th, 960th, 961st, 962nd, 963rd, 964th, 965th, 966th, 967th, 968th, 969th, 970th, 971st, 972nd, 973rd, 974th, 975th, 976th, 977th, 978th, 979th, 980th, 981st, 982nd, 983rd, 984th, 985th, 986th, 987th, 988th, 989th, 990th, 991st, 992nd, 993rd, 994th, 995th, 996th, 997th, 998th, 999th, 1000th, 1001st, 1002nd, 1003rd, 1004th, 1005th, 1006th, 1007th, 1008th, 1009th, 1010th, 1011st, 1012nd, 1013rd, 1014th, 1015th, 1016th, 1017th, 1018th, 1019th, 1020th, 1021st, 1022nd, 1023rd, 1024th, 1025th, 1026th, 1027th, 1028th, 1029th, 1030th, 1031st, 1032nd, 1033rd, 1034th, 1035th, 1036th, 1037th, 1038th, 1039th, 1040th, 1041st, 1042nd, 1043rd, 1044th, 1045th, 1046th, 1047th, 1048th, 1049th, 1050th, 1051st, 1052nd, 1053rd, 1054th, 1055th, 1056th, 1057th, 1058th, 1059th, 1060th, 1061st, 1062nd, 1063rd, 1064th, 1065th, 1066th, 1067th, 1068th, 1069th, 1070th, 1071st, 1072nd, 1073rd, 1074th, 1075th, 1076th, 1077th, 1078th, 1079th, 1080th, 1081st, 1082nd, 1083rd, 1084th, 1085th, 1086th, 1087th, 1088th, 1089th, 1090th, 1091st, 1092nd, 1093rd, 1094th, 1095th, 1096th, 1097th, 1098th, 1099th, 1100th, 1101st, 1102nd, 1103rd, 1104th, 1105th, 1106th, 1107th, 1108th, 1109th, 1110th, 1111st, 1112nd, 1113rd, 1114th, 1115th, 1116th, 1117th, 1118th, 1119th, 1120th, 1121st, 1122nd, 1123rd, 1124th, 1125th, 1126th, 1127th, 1128th, 1129th, 1130th, 1131st, 1132nd, 1133rd, 1134th, 1135th, 1136th, 1137th, 1138th, 1139th, 1140th, 1141st, 1142nd, 1143rd, 1144th, 1145th, 1146th, 1147th, 1148th, 1149th, 1150th, 1151st, 1152nd, 1153rd, 1154th, 1155th, 1156th, 1157th, 1158th, 1159th, 1160th, 1161st, 1162nd, 1163rd, 1164th, 1165th, 1166th, 1167th, 1168th, 1169th, 1170th, 1171st, 1172nd, 1173rd, 1174th, 1175th, 1176th, 1177th, 1178th, 1179th, 1180th, 1181st, 1182nd, 1183rd, 1184th, 1185th, 1186th, 1187th, 1188th, 1189th, 1190th, 1191st, 1192nd, 1193rd, 1194th, 1195th, 1196th, 1197th, 1198th, 1199th, 1200th, 1201st, 1202nd, 1203rd, 1204th, 1205th, 1206th, 1207th, 1208th, 1209th, 1210th, 1211st, 1212nd, 1213rd, 1214th, 1215th, 1216th, 1217th, 1218th, 1219th, 1220th, 1221st, 1222nd, 1223rd, 1224th, 1225th, 1226th, 1227th, 1228th, 1229th, 1230th, 1231st, 1232nd, 1233rd, 1234th, 1235th, 1236th, 1237th, 1238th, 1239th, 1240th, 1241st, 1242nd, 1243rd, 1244th, 1245th, 1246th, 1247th, 1248th, 1249th, 1250th, 1251st, 1252nd, 1253rd, 1254th, 1255th, 1256th, 1257th, 1258th, 1259th, 1260th, 1261st, 1262nd, 1263rd, 1264th, 1265th, 1266th, 1267th, 1268th, 1269th, 1270th, 1271st, 1272nd, 1273rd, 1274th, 1275th, 1276th, 1277th, 1278th, 1279th, 1280th, 1281st, 1282nd, 1283rd, 1284th, 1285th, 1286th, 1287th, 1288th, 1289th, 1290th, 1291st, 1292nd, 1293rd, 1294th, 1295th, 1296th, 1297th, 1298th, 1299th, 1300th, 1301st, 1302nd, 1303rd, 1304th, 1305th, 1306th, 1307th, 1308th, 1309th, 1310th, 1311st, 1312nd, 1313rd, 1314th, 1315th, 1316th, 1317th, 1318th, 1319th, 1320th, 1321st, 1322nd, 1323rd, 1324th, 1325th, 1326th, 1327th, 1328th, 1329th, 1330th, 1331st, 1332nd, 1333rd, 1334th, 1335th, 1336th, 1337th, 1338th, 1339th, 1340th, 1341st, 1342nd, 1343rd, 1344th, 1345th, 1346th, 1347th, 1348th, 1349th, 1350th, 1351st, 1352nd, 1353rd, 1354th, 1355th, 1356th, 1357th, 1358th, 1359th, 1360th, 1361st, 1362nd, 1363rd, 1364th, 1365th, 1366th, 1367th, 1368th, 1369th, 1370th, 1371st, 1372nd, 1373rd, 1374th, 1375th, 1376th, 1377th, 1378th, 1379th, 1380th, 1381st, 1382nd, 1383rd, 1384th, 1385th, 1386th, 1387th, 1388th, 1389th, 1390th, 1391st, 1392nd, 1393rd, 1394th, 1395th, 1396th, 1397th, 1398th, 1399th, 1400th, 1401st, 1402nd, 1403rd, 1404th, 1405th, 1406th, 1407th, 1408th, 1409th, 1410th, 1411st, 1412nd, 1413rd, 1414th, 1415th, 1416th, 1417th, 1418th, 1419th, 1420th, 1421st, 1422nd, 1423rd, 1424th, 1425th, 1426th, 1427th, 1428th, 1429th, 1430th, 1431st, 1432nd, 1433rd, 1434th, 1435th, 1436th, 1437th, 1438th, 1439th, 1440th, 1441st, 1442nd, 1443rd, 1444th, 1445th, 1446th, 1447th, 1448th, 1449th, 1450th, 1451st, 1452nd, 1453rd, 1454th, 1455th, 1456th, 1457th, 1458th, 1459th, 1460th, 1461st, 1462nd, 1463rd, 1464th, 1465th, 1466th, 1467th, 1468th, 1469th, 1470th, 1471st, 1472nd, 1473rd, 1474th, 1475th, 1476th, 1477th, 1478th, 1479th, 1480th, 1481st, 1482nd, 1483rd, 1484th, 1485th, 1486th, 1487th, 1488th, 1489th, 1490th, 1491st, 1492nd, 1493rd, 1494th, 1495th, 1496th, 1497th, 1498th, 1499th, 1500th, 1501st, 1502nd, 1503rd, 1504th, 1505th, 1506th, 1507th, 1508th, 1509th, 1510th, 1511st, 1512nd, 1513rd, 1514th, 1515th, 1516th, 1517th, 1518th, 1519th, 1520th, 1521st, 1522nd, 1523rd, 1524th, 1525th, 1526th, 1527th, 1528th, 1529th, 1530th, 1531st, 1532nd, 1533rd, 1534th, 1535th, 1536th, 1537th, 1538th, 1539th, 1540th, 1541st, 1542nd, 1543rd, 1544th, 1545th, 1546th, 1547th, 1548th, 1549th, 1550th, 1551st, 1552nd, 1553rd, 1554th, 1555th, 1556th, 1557th, 1558th, 1559th, 1560th, 1561st, 1562nd, 1563rd, 1564th, 1565th, 1566th, 1567th, 1568th, 1569th, 1570th, 1571st, 1572nd, 1573rd, 1574th, 1575th, 1576th, 1577th, 1578th, 1579th, 1580th, 1581st, 1582nd, 1583rd, 1584th, 1585th, 1586th, 1587th, 1588th, 1589th, 1590th, 1591st, 1592nd, 1593rd, 1594th, 1595th, 1596th, 1597th, 1598th, 1599th, 1600th, 1601st, 1602nd, 1603rd, 1604th, 1605th, 1606th, 1607th, 1608th, 1609th, 1610th, 1611st, 1612nd, 1613rd, 1614th, 1615th, 1616th, 1617th, 1618th, 1619th, 1620th, 1621st, 1622nd, 1623rd, 1624th, 1625th, 1626th, 1627th, 1628th, 1629th, 1630th, 1631st, 1632nd, 1633rd, 1634th, 1635th, 1636th, 1637th, 1638th, 1639th, 1640th, 1641st, 1642nd, 1643rd, 1644th, 1645th, 1646th, 1647th, 1648th, 1649th, 1650th, 1651st, 1652nd, 1653rd, 1654th, 1655th, 1656th, 1657

4.10.8 NPGA-II (Pareto)

4.10.9 NSGA-II (Pareto)

Syntax

```
[ maxfitness , minfitness , meanfitness , now_generation ,  
now_probability_crossover,best_decimal_space , best_coding_space ,error_status] =  
SGA_entry_MO_Pareto_NSGAII( options,min_confines,max_confines,probability_crossover  
,probability_mutation,population,decimal_step,max_generation,convergence_method,max  
_no_change_probability_crossover_generation,deta_fitness_max,max_probability_crossover  
_generation,max_no_change_fitness_generation,probability_crossover_step)
```

Description

In the proposed NSGA-II, we replace the sharing function approach with a crowded comparison approach that eliminates both the above difficulties to some extent. The new approach does not require any user-defined parameter for maintaining diversity among population members. Also, the suggested approach has a better computational complexity. To describe this approach, we first define a density-estimation metric and then present the crowded-comparison operator.

Examples

SGALAB_demo_MO_NSGAII.m

See Also

SGA_entry_MO_showall

SGA_entry_MO_Pareto_MOGA

SGA_entry_MO_Pareto_NPGA

SGA_entry_MO_Pareto_NSGA

SGA_entry_MO_Pareto_NSGAII

SGA_entry_MO_Pareto_PAES

SGA_entry_MO_Pareto_SPDE

SGA_entry_MO_Pareto_SPEA

SGA_entry_MO_Pareto_SPEA2

SGA_entry_MO_NON_Pareto_VEGA

SGA_entry_MO_Goal_Attainment_fitness

4.10.10 SPEA (Pareto)

4.10.11 ELSA (Pareto)

4.10.12 IEDS (Pareto)

4.10.13 MOMGA (Pareto)

4.10.14 MOMGA-II(Pareto)

4.10.15 PAES (Pareto)

4.10.16 IEDS (Pareto)

4.10.17 PDE (Pareto)

4.10.18 MPANN (Pareto)

4.10.19 SPEA2 (Pareto)

4.10.20 PCGA (Pareto)

4.10.21 SPDE (Pareto)

4.10.22 To show all the Multi-objective methods

Syntax

```
[ maxfitness , minfitness , meanfitness , now_generation ,  
now_probability_crossover,best_decimal_space , best_coding_space ,error_status] =  
SGA_entry_MO_showall( options,...  
    min_confines ,...  
    max_confines ,...  
    probability_crossover ,...  
    probability_mutation ,...  
    population ,...  
    decimal_step , ...  
    max_generation ,...  
    convergence_method ,...  
    max_no_change_probability_crossover_generation ,...  
    deta_fitness_max ,...  
    max_probability_crossover ,...  
    max_no_change_fitness_generation ,...  
    probability_crossover_step )
```

Description

to show all the multi-objective methods in SGALAB

Examples

See SGALAB_demo_MO_showall.m

See Also

SGA_entry_MO_showall

SGA_entry_MO_Pareto_MOGA

SGA_entry_MO_Pareto_NPGA

SGA_entry_MO_Pareto_NSGA

SGA_entry_MO_Pareto_NSGAII

SGA_entry_MO_Pareto_PAES

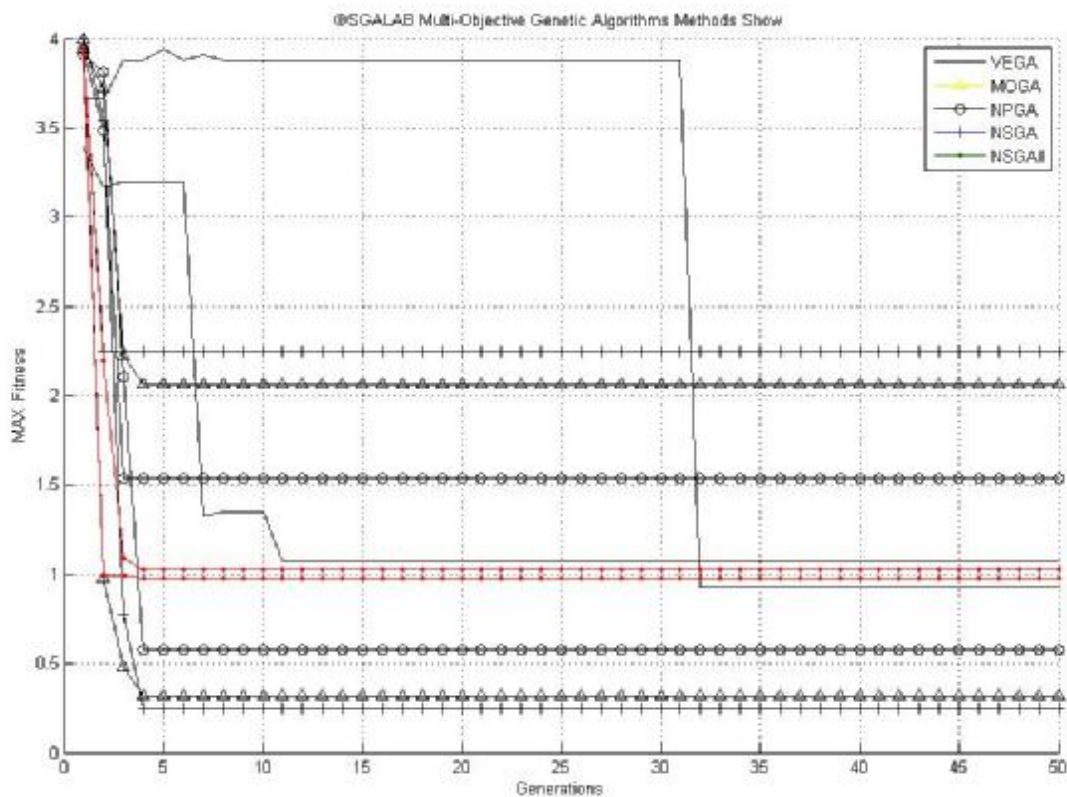
SGA_entry_MO_Pareto_SPDE

SGA_entry_MO_Pareto_SPEA

SGA_entry_MO_Pareto_SPEA2

SGA_entry_MO_NON_Pareto_VEGA

SGA_entry_MO_Goal_Attainment_fitness



4.11 Let GAs dance with other CI methods

4.11.1 Genetic Algorithms and Artificial Neural Network

4.11.2 Genetic Algorithms and Fuzzy Logic System

4.11.3 Genetic Algorithms and Ant Algorithm

4.11.4 Genetic Algorithms and Simulated Annealing Algorithm

4.11.5 Virus Evolution Genetic algorithms(VEGA)

4.11.6 Genetic Algorithms and Immune algorithm

4.12 SGALAB FAQ

4.12.1 where can I get SGALAB

See SGALAB_FAQ_howtoget.pdf

Or

You can download it from File Exchange in www.mathworks.com

4.12.2 set work path for SGALAB

See SGALAB_FAQ_setpath.pdf

4.12.3 about `randint()`

`randint()` is a standard m-function in communications toolbox of matlab, more information about `randint()`, please visit the following link:

http://www.mathworks.com/access/helpdesk_r13/help/toolbox/comm/randint.html

[Documentation](#) ▶ [Communications Toolbox](#)

Communications Toolbox



randint

Generate matrix of uniformly distributed random integers

Syntax

- `out = randint`
- `out = randint(m);`
- `out = randint(m,n);`
- `out = randint(m,n,rg);`
- `out = randint(m,n,rg,state);`

Description

`out = randint` generates a random scalar that is either 0 or 1, with equal probability.

`out = randint(m)` generates an m-by-m binary matrix, each of whose entries independently takes the value 0 with probability 1/2.

`out = randint(m,n)` generates an m-by-n binary matrix, each of whose entries independently takes the value 0 with probability 1/2.

`out = randint(m,n,rg)` generates an m-by-n integer matrix. If `rg` is zero, then `out` is a zero matrix. Otherwise, the entries are uniformly distributed and independently chosen from the range

- `[0, rg-1]` if `rg` is a positive integer
- `[rg+1, 0]` if `rg` is a negative integer
- Between `min` and `max`, inclusive, if `rg = [min,max]` or `[max,min]`

`out = randint(m,n,rg,state)` is the same as the syntax above, except that it first resets the state of the uniform random number generator rand to the integer state.

Examples

To generate a 10-by-10 matrix whose elements are uniformly distributed in the range from 0 to 7, you can use either of the following commands.

- `out = randint(10,10,[0,7]);`
- `out = randint(10,10,8);`

Syntax

Description

Examples

See Also

(To be continued...)